

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng Mô hình ngôn ngữ lớn và Tác tử AI trong
xây dựng hệ thống hỗ trợ học tập

NGUYỄN KHẮC ĐIỆP
diep.nk225806@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Ngô Văn Linh

Ngành: Công nghệ thông tin

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng Mô hình ngôn ngữ lớn và Tác tử AI trong
xây dựng hệ thống hỗ trợ học tập

NGUYỄN KHẮC ĐIỆP
diep.nk225806@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Ngô Văn Linh _____

Chữ kí GVHD

Khoa: Công nghệ thông tin Việt-Nhật

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2022

LỜI CẢM ƠN

Để hoàn thành đồ án này, em xin gửi lời cảm ơn chân thành đến quý thầy cô đã tận tâm hướng dẫn và trang bị kiến thức chuyên môn quý báu. Con xin cảm ơn gia đình vì luôn là chỗ dựa tinh thần vô giá. Cảm ơn những người bạn đồng hành và đặc biệt là người yêu đã luôn thấu hiểu, san sẻ áp lực và động viên em trong suốt thời gian qua. Và cuối cùng, tôi muốn gửi lời cảm ơn đến chính bản thân mình vì đã không bỏ cuộc, luôn chăm chỉ, kiên định và quyết tâm nỗ lực vượt qua mọi thử thách để hoàn thiện công việc với kết quả tốt nhất. Sự đồng hành của tất cả mọi người là động lực to lớn giúp em đạt được cột mốc này. Xin trân trọng cảm ơn!

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh chuyển đổi số giáo dục, học sinh và giáo viên Trung học Phổ thông (đặc biệt môn Tin học) đối mặt với nhiều thách thức. Học sinh cần môi trường học tương tác, còn giáo viên mất nhiều thời gian soạn giáo án, sinh bài tập và chấm điểm. Mặc dù các mô hình ngôn ngữ lớn (LLM) mang lại tiềm năng lớn, chúng vẫn gặp rào cản về việc sinh thông tin sai lệch (hallucination) và không bám sát chương trình chuẩn. Từ đó, bài toán đặt ra là xây dựng một trợ lý ảo giáo dục thông minh có khả năng xử lý tự động, chính xác các tác vụ dựa trên nguồn tri thức chuẩn mực.

Nghiên cứu ứng dụng kiến trúc tìm kiếm và sinh văn bản (RAG) kết hợp phương pháp điều phối đa tác nhân (Multi-Agent LLM). Cụ thể, cơ sở dữ liệu vector được xây dựng từ sách giáo khoa chuẩn. Quá trình truy xuất dữ liệu áp dụng tìm kiếm lai (từ khóa và ngữ nghĩa), tối ưu bằng thuật toán Reciprocal Rank Fusion và mạng Cross-Encoder để xếp hạng lại nhằm trích xuất chính xác ngữ cảnh. Đồng thời, hệ thống bóc tách, đánh giá và định tuyến đa ý định thông qua bộ điều khiển luồng trung tâm (Pipeline Controller). Các dịch vụ miền sau đó xử lý lệnh song song cho các tác vụ như hỏi đáp, sinh bài tập đa định dạng, chấm điểm tự luận theo tiêu chí và tạo cấu trúc bài giảng.

Thử nghiệm trên 246 mẫu kiểm thử bằng khung đo lường RAGAS cho kết quả rất khả quan. Hệ thống đạt tính trung thực (Faithfulness) 0.975 giúp khắc phục gần như hoàn toàn hiện tượng ảo giác, độ phủ ngữ cảnh (Context Recall) đạt 0.959 và mức độ liên quan (Answer Relevancy) đạt 0.857. Tốc độ phản hồi trung bình của toàn hệ thống duy trì ổn định ở mức 6,5 giây cho mỗi truy vấn phức tạp.

Nghiên cứu cung cấp một giải pháp công nghệ giáo dục (EdTech) hiệu quả, hỗ trợ tự động hóa một số quy trình dạy và học môn Tin học THPT. Đo thời, kết quả chứng minh sự thành công của kiến trúc lai RAG và Multi-Agent trong việc định hướng LLM, tạo nền tảng vững chắc để phát triển các hệ sinh thái học tập cá nhân hóa sâu rộng hơn.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

In the digital transformation of education, high school students and teachers face notable challenges in Computer Science. Students need interactive self-study environments, while teachers spend excessive time planning lessons, creating quizzes, and grading. Although Large Language Models (LLMs) offer high potential, they suffer critically from information hallucination and a lack of grounding to authorized curricula. Thus, this project proposes an automated, intelligent virtual educational assistant strictly bounded by standardized knowledge bases.

To resolve these challenges, the research introduces a system structured on a Retrieval-Augmented Generation (RAG) architecture orchestrated by a Multi-Agent LLM approach. A vector database was built indexing textbook content from grades 10 to 12. The retrieval pipeline employs a Hybrid Search strategy optimized by Reciprocal Rank Fusion and a Cross-Encoder model for precise Vietnamese document reranking. Driven by a Thin Pipeline Controller that supports dynamic multi-intent routing, domain services parallelly handle complex workflows. These include standard knowledge queries, context-aware quiz generation in varying formats, criteria-based semantic answer grading, and structured lesson plan generation.

Evaluated using the RAGAS framework across 246 comprehensive test instances, the system attained outstanding metrics. It recorded a Faithfulness score of 0.975 (virtually eliminating hallucinations), a Context Recall score of 0.959, and an Answer Relevancy score of 0.857. Furthermore, the framework demonstrated robust performance, achieving an average response latency of 6.5 seconds per multi-intent query.

Ultimately, this thesis presents a scalable EdTech solution that pragmatically automates pedagogical steps for High School Computer Science. It empirically affirms the capability of hybrid retrieval architectures coupled with multi-agent orchestration to mitigate language model hallucinations structurally. This establishes a firm foundation for deploying proactive, personalized learning ecosystems in Vietnam.

Danh mục ký hiệu và chữ viết tắt

Ký hiệu	Ý nghĩa tiếng Anh	Ý nghĩa tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
BM25	Best Matching 25	Thuật toán đối sánh tốt nhất 25
BPE	Byte-Pair Encoding	Một thuật toán mã hóa từ vựng
CD		Cánh điều (Sách giáo khoa)
CoT	Chain-of-Thought	Chuỗi tư duy
HyDE	Hypothetical Document Embeddings	Biểu diễn tài liệu giả định
IDF	Inverse Document Frequency	Tần suất nghịch đảo tài liệu
JSON	JavaScript Object Notation	Định dạng dữ liệu văn bản
KNTT		Kết nối tri thức (Sách giáo khoa)
LLM	Large Language Model	Mô hình ngôn ngữ lớn
NLP	Natural language processing	Xử lý ngôn ngữ tự nhiên
Q&A	Question and Answer	Câu hỏi và câu trả lời
RAG	Retrieval-Augmented Generation	Tạo sinh tăng cường bằng truy xuất
RRF	Reciprocal Rank Fusion	Kết hợp thứ hạng nghịch đảo
SGK		Sách giáo khoa
TF	Term Frequency	Tần suất xuất hiện của từ

Danh mục thuật ngữ chuyên ngành

Thuật ngữ	Giải thích ý nghĩa
Agent	<i>(Tác tử)</i> - Hệ thống tự trị có lỗi là trí tuệ nhân tạo, có khả năng diễn dịch yêu cầu, lập suy luận và sử dụng công cụ để thực thi hành động.
Chunk	<i>(Phân đoạn)</i> - Một đoạn văn bản có ý nghĩa hoàn chỉnh được cắt nhỏ từ tài liệu gốc, có kích thước vừa vặn với bộ nhớ của mô hình ngôn ngữ.
Chunking	<i>(Phân mảnh dữ liệu)</i> - Kỹ thuật chia nhỏ các tài liệu quy mô lớn thành các chunk nhằm thuận tiện cho việc lập chỉ mục và tìm kiếm thông tin.
Context Window	<i>(Cửa sổ ngữ cảnh)</i> - Giới hạn tối đa số lượng từ (hoặc token) mà một LLM có thể tiếp nhận và ghi nhớ trong một phiên xử lý duy nhất.
Embedding	<i>(Biểu diễn Vector/Nhúng)</i> - Kỹ thuật mã hóa ngôn ngữ tự nhiên thành các vector số học có độ phân giải cao, giúp máy tính có thể tính toán đo lường mức độ tương đồng ý nghĩa.
Hallucination	<i>(Hiện tượng Ảo giác)</i> - Trạng thái mà mô hình ngôn ngữ lớn đưa ra những câu trả lời có hệ thống về ngoài rất tự tin nhưng hoàn toàn sai sự thật hoặc thiếu căn cứ.
Metadata	<i>(Siêu dữ liệu)</i> - Các thẻ thông tin đi kèm (như Tên bài học, Chương, Chủ đề) nhằm bổ trợ và cung cấp căn cứ nguồn gốc cho các chunk dữ liệu.
Multi-Agent	<i>(Đa tác tử)</i> - Một hệ thống phức hợp nơi nhiều tác tử (agent) hoạt động song song hoặc phối hợp nhằm giải tỏa áp lực của một tác vụ quá lớn.
Prompt	<i>(Chỉ thị)</i> - Câu lệnh hoặc ngữ cảnh đầu vào hướng dẫn mô hình ngôn ngữ lớn thực hiện trả lời, thiết lập phong cách hay xuất ra bằng định dạng nhất định.
Reranker	<i>(Bộ xếp hạng lại)</i> - Mô hình máy học nâng cao làm nhiệm vụ rà soát, tinh chỉnh thứ tự các kết quả tìm ráp ban đầu để đẩy văn bản có liên đới chặt chẽ nhất lên cực đỉnh.

Retriever	<i>(Bộ truy xuất)</i> - Thành phần thuật toán thực hiện tìm kiếm, quét qua toàn bộ cơ sở dữ liệu Vector để lấy ra cụm tài liệu thô gần khớp với truy vấn người dùng.
Token	<i>(Đơn vị cơ sở)</i> - Thành phần nhỏ nhất của một đoạn văn bản (có thể là một từ, âm tiết, hoặc cụm ký tự) mà máy tính có thể tách ra để xử lý.
Vector Database	<i>(Cơ sở dữ liệu Vector)</i> - Cơ sở dữ liệu chuyên dụng được thiết kế nhằm lưu giữ và truy vấn tốc độ cao đối với dữ liệu kiểu vector đa chiều.
Underthesea	<i>(Thư viện NLP tiếng Việt)</i> - Một bộ công cụ mã nguồn mở phụ vụ mạnh mẽ cho nhiều bài toán xử lý ngôn ngữ tự nhiên tiếng Việt, nổi bật là tách từ (Word Segmentation).
Word Segmentation	<i>(Tách từ)</i> - Kỹ thuật phân tách chuỗi văn bản thành hệ thống các từ vựng hợp cấu trúc, đặc biệt quan trọng với ngôn ngữ không có định danh ranh giới sẵn bằng khoảng trắng (ví dụ từ ghép tiếng Việt).

MỤC LỤC

Danh mục ký hiệu và chữ viết tắt.....	i
Danh mục thuật ngữ chuyên ngành	ii
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT, KHẢO SÁT VÀ ĐỀ XUẤT GIẢI PHÁP.....	6
2.1 Phạm vi nghiên cứu	6
2.2 Các nghiên cứu liên quan	7
2.3 Kiến thức nền tảng về RAG cho hỏi đáp dựa trên tài liệu	8
2.4 Sinh tăng cường truy xuất.....	9
2.5 Kiến thức nền tảng về truy xuất, xếp hạng lại và đánh giá.....	10
2.5.1 Lọc metadata theo phạm vi	10
2.5.2 Viết lại truy vấn và truy xuất đa truy vấn.....	10
2.5.3 Xếp hạng lại	10
2.5.4 Truy xuất thích nghi	11
2.6 Kết chương.....	11
CHƯƠNG 3. THIẾT KẾ GIẢI PHÁP VÀ PHƯƠNG PHÁP ĐÁNH GIÁ. 13	13
3.1 Mục tiêu thiết kế.....	13
3.2 Kiến trúc tổng thể của giải pháp.....	13
3.3 Thiết kế dữ liệu đầu vào	14
3.4 Thiết kế tiền xử lý và chunking	15

3.5 Thiết kế embedding và indexing.....	15
3.6 Thiết kế phân tích ngữ cảnh, intent và phạm vi	15
3.7 Thiết kế retrieval service	16
3.7.1 Vector search cơ bản.....	16
3.7.2 Vector search có filter phạm vi	16
3.7.3 Search với query rewriting	16
3.7.4 Multi-query retrieval và merge kết quả.....	17
3.7.5 Reranking.....	17
3.7.6 Adaptive retrieval.....	17
3.8 Thiết kế context building và generation.....	17
3.9 Thiết kế benchmark retrieval.....	18
3.10 Thiết kế đánh giá retrieval	18
3.11 Thiết kế đánh giá generation	19
3.12 Thiết kế logging, session và trace	20
3.13 Kết chương.....	20
CHƯƠNG 4. CÀI ĐẶT, KIỂM THỬ VÀ KẾT QUẢ THỰC NGHIỆM.....	21
4.1 Môi trường và dữ liệu thực nghiệm.....	21
4.1.1 Môi trường phát triển	21
4.1.2 Dữ liệu thực nghiệm.....	21
4.2 Tổng quan kiến trúc toàn bộ hệ thống	22
4.3 Thiết kế chi tiết các thành phần hệ thống.....	23
4.3.1 Thiết kế Lớp Ngữ cảnh và Ý định (Context & Intent Layer)	23
4.3.2 Thiết kế Lớp Lập kế hoạch (Planning Layer).....	25
4.3.3 Thiết kế Lớp Thực thi (Execution Layer)	25
4.3.4 Thiết kế Quy trình Đa tác tử cho Slide và Giáo án	26
4.3.5 Thiết kế Lớp Tri thức (Knowledge Layer)	28

4.3.6 Thiết kế Lớp Sinh phản hồi và Hậu kiểm (Generation & Validation).....	29
4.3.7 Thiết kế Trả phản hồi, Lưu phiên và Ghi vết	30
4.4 Đánh giá hiệu năng và Kết quả thực nghiệm.....	32
4.4.1 Phương pháp đánh giá	32
4.4.2 Kết quả thực nghiệm phân hệ RAG	32
4.4.3 Đánh giá độ trễ hệ thống (Latency).....	33
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT	35
5.1 Điều phối request bằng thin orchestrator	35
5.1.1 Vấn đề cần giải quyết.....	35
5.1.2 Giải pháp	35
5.1.3 Đóng góp đạt được	36
5.2 Truy xuất lai và Adaptive RAG	36
5.2.1 Vấn đề cần giải quyết.....	36
5.2.2 Giải pháp	36
5.2.3 Đóng góp đạt được	37
5.3 Multi-intent routing và action planning.....	37
5.3.1 Vấn đề cần giải quyết.....	37
5.3.2 Giải pháp	37
5.3.3 Đóng góp đạt được	38
5.4 Tách quiz standalone khỏi content assessment	38
5.4.1 Vấn đề cần giải quyết.....	38
5.4.2 Giải pháp	38
5.4.3 Đóng góp đạt được	39
5.5 Content multi-agent cho slide và giáo án.....	39
5.5.1 Vấn đề cần giải quyết.....	39

5.5.2 Giải pháp	39
5.5.3 Đóng góp đạt được	39
5.6 Human-in-the-loop tại bước duyệt dần ý	40
5.6.1 Vấn đề cần giải quyết.....	40
5.6.2 Giải pháp	40
5.6.3 Đóng góp đạt được	40
5.7 Quality path, session và trace.....	40
5.7.1 Vấn đề cần giải quyết.....	40
5.7.2 Giải pháp	40
5.7.3 Đóng góp đạt được	41
5.8 Tool layer như điểm mở rộng có kiểm soát	41
5.8.1 Vấn đề cần giải quyết.....	41
5.8.2 Giải pháp	41
5.8.3 Đóng góp đạt được	41
5.9 Kết chương.....	41
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	43
6.1 Kết luận.....	43
6.2 Hướng phát triển.....	43
TÀI LIỆU THAM KHẢO.....	46
PHỤ LỤC.....	48
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP	48
A.1 Ngành học.....	49
A.2 Đánh dấu (bullet) và đánh số (numering)	49
A.3 Cách thêm bảng	50
A.4 Chèn hình ảnh	50
A.5 Tài liệu tham khảo	51

A.6 Cách viết phương trình và công thức toán học.....	51
A.7 Qui cách đóng quyển.....	51
B. ĐẶC TẢ USE CASE.....	53
B.1 Đặc tả use case “Thông kê tình hình mượn sách”	53
B.2 Đặc tả use case “Đăng ký làm thẻ mượn”	53

DANH MỤC HÌNH VẼ

Hình 3.1	Tổng quan pipeline dữ liệu, pipeline truy vấn và hai luồng đánh giá	14
Hình 4.1	Sơ đồ tổng quan kiến trúc hệ thống theo vòng đời xử lý yêu cầu	23
Hình 4.2	Chi tiết cấu trúc và luồng xử lý tại Lớp Ngữ cảnh và Ý định . .	24
Hình 4.3	Chu trình lập kế hoạch hành động bằng rule-based logic . . .	25
Hình 4.4	Sơ đồ vòng lặp đa tác vụ và điều phối dịch vụ nghiệp vụ . . .	26
Hình 4.5	Pipeline đa tác tử nội bộ cho sinh slide và giáo án	27
Hình 4.6	Cấu trúc xử lý và truy xuất thông tin của Knowledge Layer . .	28
Hình 4.7	Hai đường sinh và hậu kiểm chất lượng trong hệ thống	29
Hình 4.8	Khối phản hồi, ghi trace và lưu trạng thái phiên	31
Hình A.1	Internet vạn vật	50
Hình A.2	Quy cách đóng quyển đồ án	52
Hình A.3	Quy cách đóng quyển đồ án	52

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các nhóm giải pháp cho bài toán chatbot giáo dục . . .	8
Bảng 3.1	Các trường metadata chính của chunk	14
Bảng 3.2	Cấu trúc đề xuất cho một mẫu retrieval benchmark	18
Bảng 3.3	Rubric chấm generation theo thang 1-5	19
Bảng 4.1	Kết quả đánh giá hệ thống theo framework Ragas	32
Bảng 4.2	Thống kê thời gian trễ của các khối kiến trúc trong hệ thống .	33
Bảng A.1	Table to test captions and labels.	50

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh cuộc cách mạng công nghiệp lần thứ tư đang diễn ra mạnh mẽ trên toàn cầu, chuyển đổi số đã trở thành một trong những ưu tiên hàng đầu của Việt Nam trong chiến lược phát triển quốc gia. Giáo dục được xác định là một trong những lĩnh vực trọng điểm trong quá trình chuyển đổi số, việc ứng dụng công nghệ thông tin và trí tuệ nhân tạo (AI) vào dạy và học không chỉ là xu hướng tất yếu mà còn là yêu cầu cấp thiết để nâng cao chất lượng giáo dục.

Môn Tin học tại trường trung học phổ thông (THPT) đóng vai trò đặc biệt quan trọng trong chương trình giáo dục Việt Nam. Với chương trình mới bao gồm hai bộ sách giáo khoa Cánh Diều và Kết Nối Tri Thức với Cuộc Sống cho các lớp 10, 11 và 12, nội dung bao phủ nhiều chủ đề chuyên môn từ khoa học máy tính, mạng máy tính đến an toàn thông tin. Tuy nhiên, việc giảng dạy và học tập môn học này vẫn đối mặt với nhiều thách thức đáng kể.

Về phía học sinh, các em thường thiếu một công cụ hỗ trợ tự học hiệu quả và bám sát nội dung sách giáo khoa. Các chatbot AI phổ quát hiện nay như ChatGPT, Claude hay Gemini tuy có khả năng trả lời câu hỏi nhưng không được tối ưu cho dữ liệu sách giáo khoa Việt Nam, dẫn đến tình trạng đưa ra thông tin không chính xác (hallucination). Điều này đặc biệt nguy hiểm trong bối cảnh giáo dục khi mà việc tiếp thu kiến thức sai lệch có thể ảnh hưởng nghiêm trọng đến quá trình học tập. Bên cạnh đó, học sinh cũng thiếu nguồn bài tập đa dạng và phản hồi tức thời được thiết kế chuyên biệt theo từng bài học.

Về phía giáo viên, việc biên soạn tài liệu giảng dạy như câu hỏi trắc nghiệm, bài tập hay cấu trúc slide bài giảng đòi hỏi rất nhiều thời gian và công sức. Trong khi đó, hầu hết các nền tảng e-learning hiện có tại Việt Nam chỉ đóng vai trò là kho lưu trữ và phân phối bài giảng số, chưa có khả năng tự động khởi tạo nội dung thông minh dựa trên yêu cầu của người dùng.

Từ những phân tích trên, nhu cầu xây dựng một hệ thống trợ lý ảo giáo dục ứng dụng công nghệ Tăng cường truy xuất dữ liệu (RAG) sử dụng dữ liệu nền tảng là sách giáo khoa trở nên cấp thiết. Hệ thống này cần đáp ứng đầy đủ các mục tiêu bao gồm tự động trả lời câu hỏi dựa trên nội dung SGK, sinh bài tập đa dạng định dạng, phát thảo cấu trúc slide bài giảng và hỗ trợ giáo viên trong công tác soạn giảng, qua đó tối ưu hóa công việc của cả người dạy lẫn người học.

1.2 Mục tiêu và phạm vi đề tài

Trong những năm gần đây, việc ứng dụng trí tuệ nhân tạo vào giáo dục đã thu hút sự quan tâm của nhiều nhà nghiên cứu và doanh nghiệp trên thế giới. Các mô hình ngôn ngữ lớn (LLM) như ChatGPT, Claude hay Gemini đã chứng minh khả năng ấn tượng trong việc tạo sinh văn bản và trả lời câu hỏi. Tuy nhiên, khi được sử dụng trong bối cảnh giáo dục cụ thể, các mô hình này bộc lộ những hạn chế đáng kể. Đầu tiên, chúng không được huấn luyện trên dữ liệu sách giáo khoa Việt Nam, dẫn đến thường xuyên đưa ra câu trả lời không chính xác hoặc không phù hợp với nội dung chương trình học. Thứ hai, hiện tượng hallucination - tức là sinh ra thông tin nhìn có vẻ đáng tin cậy nhưng hoàn toàn sai sự thật - là vấn đề nan giải mà các LLM hiện tại vẫn chưa thể khắc phục triệt để.

Đối với các nền tảng e-learning trong nước như VnEdu, HOC247, hay Moon, các hệ thống này chủ yếu cung cấp kho tài liệu số và video bài giảng được biên soạn sẵn. Một số nền tảng đã tích hợp tính năng quiz tự động nhưng chưa có khả năng sinh câu hỏi theo yêu cầu cụ thể của người dùng hoặc dựa trên nội dung bài học mới được cập nhật. Việc quản lý và cập nhật ngân hàng câu hỏi vẫn phải thực hiện thủ công, tốn kém nhiều thời gian và công sức.

Về mặt kỹ thuật, công nghệ Retrieval-Augmented Generation (RAG) đã được chứng minh hiệu quả trong việc cải thiện độ chính xác của LLM bằng cách kết hợp khả năng truy xuất tài liệu với khả năng sinh ngôn ngữ tự nhiên. Nhiều nghiên cứu đã áp dụng RAG vào các bài toán giáo dục và đạt được những kết quả khả quan. Tuy nhiên, phần lớn các ứng dụng này tập trung vào các ngữ cảnh sử dụng tiếng Anh và chưa có nhiều nghiên cứu cụ thể cho việc xây dựng hệ thống RAG dựa trên sách giáo khoa Việt Nam. Đặc biệt, việc kết hợp kiến trúc Multi-Agent với RAG để tạo ra hệ thống đa tác vụ (quiz + slide + scoring) vẫn còn là lĩnh vực tương đối mới và chưa được khai thác nhiều trong bối cảnh giáo dục Việt Nam.

So sánh giữa các giải pháp hiện có cho thấy rõ sự khác biệt về năng lực. Trong khi các chatbot AI phổ quát có khả năng đối thoại tự do nhưng không đảm bảo tính chính xác theo SGK, các nền tảng e-learning trong nước có nguồn dữ liệu chuẩn nhưng thiếu khả năng tự động sáng tạo nội dung. Đề tài này hướng tới việc kết hợp ưu điểm của cả hai hướng tiếp cận: đảm bảo tính bám sát sách giáo khoa đồng thời có khả năng sinh tự động các nội dung giáo dục đa dạng.

Dựa trên các phân tích và đánh giá ở trên, đề tài này xác định các hạn chế cần khắc phục bao gồm: thiếu công cụ AI tối ưu cho việc học Tin học THPT tại Việt Nam; chưa có giải pháp RAG nào tích hợp đầy đủ các tác vụ quiz, slide và scoring; và chưa có hệ thống nào kết hợp Multi-Agent với cơ chế Human-In-The-

Loop (HITL) để đảm bảo chất lượng đầu ra trong việc sinh slide bài giảng.

Mục tiêu tổng quát của đề tài là xây dựng hệ thống trợ lý ảo giáo dục hỗ trợ việc dạy và học môn Tin học tại trường trung học phổ thông, sử dụng nền tảng là sách giáo khoa Tin học lớp 10, 11 và 12 thuộc hai bộ sách Cánh Diều và Kết Nối Tri Thức.

Các mục tiêu cụ thể bao gồm: nghiên cứu và xây dựng hệ thống truy xuất kiến thức từ sách giáo khoa dựa trên kiến trúc RAG; thiết kế và phát triển module sinh câu hỏi và bài tập tự động với nhiều định dạng bao gồm trắc nghiệm, tự luận, điền khuyết và đúng/sai; xây dựng module sinh cấu trúc slide bài giảng với kiến trúc đa tác tử; phát triển module chấm điểm và phản hồi tự động dựa trên ngữ cảnh bài học; cuối cùng là đánh giá hiệu năng hệ thống thông qua các chỉ số đánh giá chuẩn.

1.3 Định hướng giải pháp

Để giải quyết các vấn đề đã xác định ở phần trên, đề tài này lựa chọn định hướng giải pháp dựa trên sự kết hợp giữa công nghệ Retrieval-Augmented Generation (RAG) và kiến trúc Multi-Agent. Đây là hai công nghệ tiên tiến trong lĩnh vực AI hiện nay, có khả năng bổ trợ lẫn nhau để tạo ra một hệ thống vừa đảm bảo tính chính xác vừa có khả năng xử lý đa tác vụ phức tạp.

Về nền tảng LLM, hệ thống sử dụng mô hình ngôn ngữ lớn của Google với ưu điểm là chi phí thấp, khả năng hỗ trợ tiếng Việt tốt và chế độ sinh output có cấu trúc ổn định. Mô hình này được đánh giá là phù hợp với mục tiêu của đề tài khi cần tối ưu hóa chi phí trong khi vẫn đảm bảo chất lượng đầu ra cần thiết.

Về thành phần truy xuất, thay vì phụ thuộc vào các thư viện đóng gói, đề tài tự xây dựng bộ máy truy xuất kết hợp giữa thuật toán BM25 và tìm kiếm ngữ nghĩa (Semantic Search). Hai luồng kết quả này được hợp nhất thông qua thuật toán Reciprocal Rank Fusion (RRF). Cách tiếp cận này mang lại sự chủ động trong việc tối ưu hóa thuật toán cho văn bản tiếng Việt và tránh phụ thuộc vào các thư viện có thể gây tốn kém tài nguyên không cần thiết.

Về kiến trúc điều phối, hệ thống thiết kế một Orchestrator tại lớp mức lệnh cơ sở thay vì sử dụng các framework có sẵn. Orchestrator này đóng vai trò là bộ xử lý mỏng, điều phối các thành phần xử lý bao gồm ContextAnalyzer, SessionManager, ActionPlanner và RAG Agent. Ưu điểm của cách thiết kế này là toàn bộ quy trình xử lý được tối ưu về tốc độ, đồng thời quá trình theo vết và debug trở nên dễ dàng hơn.

Về kiến trúc Multi-Agent, hệ thống áp dụng mô hình Supervisor với Supervisor Agent đóng vai trò điều phối, tiếp nhận yêu cầu và phân loại đến các sub-agents

chuyên biệt bao gồm: OutlineAgent cho việc tạo dàn ý bài giảng, ContentAgent cho việc sinh nội dung chi tiết, MediaAgent cho việc tìm kiếm hình ảnh minh họa, QuizAgent cho việc sinh câu hỏi luyện tập, và MergeAgent cho việc tổng hợp kết quả. Hệ thống còn tích hợp cơ chế Human-In-The-Loop (HITL) cho phép người dùng duyệt và chỉnh sửa outline trước khi tiến hành sinh nội dung chi tiết.

Về đóng góp chính, đề tài này tự xây dựng bộ máy truy xuất từ mức cơ sở, thiết kế kiến trúc Adaptive RAG với nhiều chiến lược truy xuất khác nhau, xây dựng hệ thống Multi-Agent cho sinh slide bài giảng với các sub-agents chuyên biệt, và tích hợp cơ chế HITL trong quy trình sinh slide.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày về việc khảo sát thực trạng và phân tích yêu cầu của hệ thống trợ lý ảo giáo dục. Trong chương này, em giới thiệu các tác nhân tham gia vào hệ thống, các Use case tổng quan và Use case phân rã, đồng thời đặc tả chi tiết các luồng chức năng chính bao gồm Hỏi đáp kiến thức SGK, Sinh câu hỏi và bài tập, Chấm điểm và phản hồi, và Sinh slide bài giảng.

Chương 3 giới thiệu về các công nghệ và kiến thức nền tảng được áp dụng trong đồ án. Em trình bày các khái niệm về Trí tuệ nhân tạo (AI) và Mô hình ngôn ngữ lớn (LLM), thuật toán Tokenization và Embedding, kiến trúc Transformer và cơ chế Attention. Ngoài ra, chương này cũng phân tích chi tiết công nghệ RAG bao gồm các thuật toán truy xuất, tìm kiếm lai, cùng với kiến trúc Multi-Agent và các phương pháp đánh giá hệ thống.

Chương 4 trình bày về việc thiết kế và cài đặt hệ thống. Em mô tả kiến trúc tổng thể của hệ thống với nhiều lớp xử lý bao gồm Context & Intent Layer, Planning Layer, Execution Layer, Knowledge Layer, Generation Layer và Response & Memory. Chương này trình bày chi tiết từng thành phần từ ContextAnalyzer, QueryRewriter, IntentRouter, SessionManager, ActionPlanner, cho đến RAG Service với nhiều chiến lược truy xuất, Quiz Service và Slide Service với kiến trúc Multi-Agent Supervisor.

Chương 5 trình bày về các giải pháp kỹ thuật đóng góp chính của đồ án. Em giới thiệu chi tiết về việc tự xây dựng bộ máy truy xuất từ mức cơ sở, thiết kế kiến trúc Orchestrator code-level, mô hình Adaptive RAG Agent với suy luận Heuristics, 2-Level Intent Detection, chiến lược quản lý Memory State, và hệ thống Multi-Agent với Supervisor pattern cho sinh slide bài giảng.

Cuối cùng, phần Kết luận tổng kết các kết quả đạt được của đồ án, đánh giá các

chỉ số hiệu năng, đồng thời đề xuất các hạn chế và hướng phát triển trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT, KHẢO SÁT VÀ ĐỀ XUẤT GIẢI PHÁP

Chương 1 đã trình bày bối cảnh, mục tiêu và phạm vi tổng quát của đề tài. Từ cơ sở đó, chương này tập trung phân tích rõ ngữ cảnh nghiên cứu, các hướng tiếp cận liên quan và những kiến thức nền cần thiết cho đề án. Mục tiêu của chương là làm rõ bài toán đang được giải quyết, chỉ ra khoảng trống của các nghiên cứu hoặc hệ thống gần với đề tài, đồng thời cung cấp nền tảng lý thuyết vừa đủ để hiểu các quyết định thiết kế ở các chương sau.

Nội dung chương được tổ chức thành bốn phần. Phần đầu trình bày phạm vi nghiên cứu và ngữ cảnh bài toán. Phần thứ hai khảo sát các hướng tiếp cận liên quan dưới góc nhìn ưu điểm, hạn chế và mức độ phù hợp với dữ liệu SGK. Hai phần tiếp theo tập trung vào các kiến thức nền có liên hệ trực tiếp đến đề án, gồm RAG cho hỏi đáp dựa trên tài liệu và các kỹ thuật retrieval, reranking cùng phương pháp đánh giá.

2.1 Phạm vi nghiên cứu

Chatbot giáo dục trong phạm vi đề án này được hiểu là một hệ thống hỗ trợ người học và giáo viên tương tác với tri thức môn Tin học THPT bằng ngôn ngữ tự nhiên. Khác với chatbot hội thoại thông thường, hệ thống cần trả lời dựa trên nội dung sách giáo khoa, đồng thời cần tôn trọng phạm vi lớp học, bộ sách và chủ đề mà người dùng đang quan tâm.

Dữ liệu của đề tài gồm SGK Tin học THPT lớp 10, 11 và 12 thuộc hai bộ Cánh Diều và Kết Nối Tri Thức. Các tài liệu này có cấu trúc phân cấp theo bộ sách, lớp, chủ đề, bài học và mục nội dung. Đặc điểm phân cấp này tạo ra lợi thế khi truy xuất theo phạm vi, nhưng cũng đặt ra yêu cầu hệ thống phải quản lý metadata chính xác. Nếu metadata bị bỏ qua, câu trả lời có thể trộn nội dung giữa hai bộ sách hoặc giữa các lớp khác nhau.

Với người học, nhu cầu chính là đặt câu hỏi tự nhiên, yêu cầu giải thích khái niệm, ôn lại nội dung bài học hoặc luyện tập theo chủ đề. Với giáo viên, hệ thống có thể hỗ trợ chuẩn bị câu hỏi, dàn ý slide hoặc giáo án. Tuy nhiên, các tác vụ này đều phụ thuộc vào một nền tảng chung: hệ thống phải truy xuất đúng ngữ cảnh từ SGK trước khi sinh nội dung.

Hệ thống hỏi đáp có thể được chia thành hai nhóm chính: closed-book question answering và open-book question answering. Trong closed-book question answering, mô hình trả lời chủ yếu dựa trên tri thức đã học trong quá trình huấn luyện. Trong open-book hoặc document-grounded question answering, hệ thống

được cung cấp thêm tài liệu nguồn và phải dựa vào tài liệu đó để trả lời.

Đề tài này thuộc nhóm document-grounded question answering. Câu trả lời không nên chỉ dựa vào tri thức tham số của mô hình ngôn ngữ, vì tri thức đó có thể không khớp với chương trình SGK Việt Nam hoặc không phản ánh đúng phạm vi sách người dùng đang học. Thay vào đó, hệ thống cần tìm các đoạn tài liệu liên quan trong SGK, sau đó dùng chúng làm ngữ cảnh để sinh câu trả lời.

Một pipeline hỏi đáp dựa trên tài liệu thường gồm các bước chính: tiếp nhận câu hỏi, xác định phạm vi hoặc ý định, truy xuất tài liệu liên quan, xây dựng ngữ cảnh, sinh câu trả lời và kiểm tra kết quả. Trong pipeline này, bước truy xuất đóng vai trò đặc biệt quan trọng. Nếu hệ thống truy xuất sai hoặc thiếu ngữ cảnh, mô hình sinh câu trả lời khó có thể tạo ra kết quả đúng, dù prompt được thiết kế tốt.

2.2 Các nghiên cứu liên quan

Các nghiên cứu và giải pháp gần với đề tài hiện nay có thể được nhìn theo cả góc độ học thuật lẫn góc độ sản phẩm. Trong phạm vi bài toán hỗ trợ học tập môn Tin học THPT theo chương trình GDPT 2018, ba nhóm đối tượng đáng chú ý nhất gồm các mô hình ngôn ngữ lớn đa mục đích, các chatbot dịch vụ trong nước và các hệ thống học tập hoặc đồ án nghiên cứu đi trước.

2.2.1 Nhóm mô hình ngôn ngữ lớn đa mục đích

Nhóm này bao gồm các nền tảng như ChatGPT, Gemini và Claude. Ưu điểm lớn nhất của chúng là khả năng hội thoại tự nhiên, suy luận linh hoạt và hỗ trợ tốt các câu hỏi liên quan đến mã nguồn. Với các bài toán như giải thích một đoạn chương trình hoặc gợi ý thuật toán tổng quát, các mô hình này thường cho chất lượng phản hồi cao.

Tuy nhiên, hạn chế quan trọng khi áp dụng trực tiếp vào giáo dục phổ thông Việt Nam là thiếu ngữ cảnh cục bộ. Do được huấn luyện trên dữ liệu toàn cầu, các mô hình này không tự nhận biết được ranh giới kiến thức theo lớp học, bộ sách và định hướng môn học trong chương trình GDPT 2018. Khi học sinh hỏi một khái niệm cơ bản, hệ thống có thể trả lời bằng nội dung vượt cấp hoặc theo quy chuẩn ngoài phạm vi SGK. Ngoài ra, các nội dung mang tính đặc thù của chương trình Tin học Việt Nam, như cách diễn giải lý thuyết, định nghĩa trong SGK hoặc các chủ đề về đạo đức số và pháp luật trong môi trường số, có nguy cơ bị trả lời lệch khỏi yêu cầu sư phạm.

2.2.2 Nhóm chatbot dịch vụ và quản trị trong nước

Nhóm này có thể đại diện bởi các nền tảng chatbot do doanh nghiệp trong nước phát triển, chẳng hạn các giải pháp của FPT hoặc VNPT. Điểm mạnh của nhóm

này là tính ổn định, khả năng tích hợp vào hệ thống tổ chức và mức độ tối ưu hóa cho tiếng Việt. Các hệ thống như vậy thường phù hợp với những bài toán tư vấn dịch vụ, tự động hóa quy trình hành chính hoặc hỗ trợ tra cứu thông tin có cấu trúc.

Tuy nhiên, khi xét dưới góc độ hỗ trợ học tập học thuật, các chatbot dịch vụ này chưa phải là lời giải phù hợp. Chúng thường không có module phân tích mã nguồn, không có cơ chế truy xuất tri thức học liệu theo bài học, và cũng không được thiết kế để hướng dẫn người học theo từng bước tư duy. Do đó, dù có lợi thế về hạ tầng và tiếng Việt, nhóm giải pháp này chưa đáp ứng trực tiếp yêu cầu của một trợ lý học tập cho môn Tin học THPT.

2.2.3 Nhóm các hệ thống nghiên cứu và đồ án đi trước

Trong môi trường học thuật, đã có những hệ thống thử nghiệm hỗ trợ học tập bằng chatbot hoặc hỏi đáp tự động. Ưu điểm của nhóm này là tiếp cận gần hơn với đối tượng người dùng cuối và thường bám vào giáo trình hoặc dữ liệu nội địa. Một số hệ thống cũng đã bắt đầu khai thác mô hình ngôn ngữ lớn để sinh dữ liệu hoặc sinh phản hồi.

Dù vậy, nhiều hệ thống đi trước vẫn dựa mạnh vào kiến trúc rule-based hoặc intent classification kết hợp với cơ sở dữ liệu quan hệ truyền thống. Cách làm này dễ kiểm soát nhưng kém linh hoạt khi học sinh dùng từ đồng nghĩa, thay đổi cấu trúc câu hoặc hỏi lệch khỏi bộ kịch bản đã xây dựng. Với các hệ thống có dùng LLM, không ít trường hợp chỉ dừng ở mức gọi API trực tiếp và thiết kế prompt thủ công, nhưng chưa có cơ chế retrieval chuẩn hóa từ học liệu nguồn. Hạn chế của hướng này là chi phí API cao, phản hồi thiếu ổn định và nguy cơ câu trả lời trôi khỏi mạch kiến thức SGK.

Bảng 2.1: So sánh các nhóm giải pháp cho bài toán chatbot giáo dục

Nhóm giải pháp	Hiểu câu hỏi tự nhiên	Dùng dữ liệu SGK riêng	Khả năng giảm hallucination
Rule-based	Thấp	Có, nhưng khó mở rộng	Cao nếu luật đầy đủ
FAQ/keyword search	Thấp-Trung bình	Có	Trung bình
Semantic search	Trung bình	Có	Phụ thuộc tài liệu trả về
LLM trực tiếp	Cao	Thấp nếu không nạp context	Thấp-Trung bình
RAG	Cao	Cao	Cao hơn nếu retrieval tốt
Adaptive RAG	Cao	Cao	Cao hơn nhờ chọn chiến lược theo câu hỏi

Nhìn từ bảng so sánh, các mô hình LLM đa mục đích có ưu thế rõ rệt về năng lực diễn giải và xử lý mã nguồn, nhưng thiếu kiểm soát theo phạm vi SGK. Các chatbot doanh nghiệp trong nước mạnh về hạ tầng và xử lý tiếng Việt, nhưng không được thiết kế cho bài toán gia sư học thuật. Các hệ thống nghiên cứu đi trước tiếp cận gần hơn với bối cảnh giáo dục, song nhiều hệ thống vẫn còn cứng nhắc hoặc chưa có cơ chế retrieval đủ chặt để neo câu trả lời vào học liệu chuẩn.

Từ đó có thể thấy khoảng trống nghiên cứu của đề tài nằm ở ba điểm. Thứ nhất, cần một hệ thống vừa tận dụng được khả năng diễn giải của LLM, vừa kiểm soát được tri thức theo phạm vi SGK. Thứ hai, hệ thống cần hỗ trợ truy xuất theo lớp, bộ sách và chủ đề thay vì trả lời theo tri thức toàn cục. Thứ ba, chất lượng của hệ thống cần được đánh giá theo từng node, đặc biệt là retrieval và generation, thay vì chỉ nhìn vào câu trả lời cuối cùng. Đây chính là động lực để đề án lựa chọn hướng RAG có xét phạm vi dữ liệu, có khả năng thử nhiều cấu hình search và có benchmark đánh giá riêng cho retrieval.

2.3 Kiến thức nền tảng về RAG cho hỏi đáp dựa trên tài liệu

Mô hình ngôn ngữ lớn là thành phần có khả năng sinh văn bản tự nhiên, giải thích khái niệm và chuyển đổi yêu cầu của người dùng thành đầu ra có cấu trúc. Cơ chế sinh của LLM dựa trên việc dự đoán token tiếp theo theo ngữ cảnh đã có:

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1}) \quad (2.1)$$

Trong đó, w_t là token tại vị trí t . Cơ chế này giúp LLM tạo câu trả lời mạch lạc về mặt ngôn ngữ, nhưng không bảo đảm rằng mọi thông tin được sinh ra đều có căn cứ trong tài liệu đang xét.

Khi dùng trực tiếp LLM cho bài toán giáo dục, có một số hạn chế chính. Thứ nhất, mô hình có thể không biết hoặc không ưu tiên đúng nội dung trong SGK Tin học Việt Nam. Thứ hai, mô hình có thể trả lời bằng tri thức ngoài phạm vi bài học, dẫn đến câu trả lời không phù hợp với chương trình. Thứ ba, hiện tượng hallucination có thể làm hệ thống sinh thông tin nghe hợp lý nhưng không tồn tại trong tài liệu. Thứ tư, context window của mô hình có giới hạn, nên không thể đưa toàn bộ SGK vào prompt cho mọi câu hỏi.

Vì các lý do trên, LLM trong đề án không được sử dụng như nguồn tri thức duy nhất. Mô hình được đặt sau lớp truy xuất, nhận ngữ cảnh từ SGK và sinh câu trả lời dựa trên ngữ cảnh đó. Cách tổ chức này giúp giảm phụ thuộc vào tri thức tham số của mô hình và tạo điều kiện để đánh giá lỗi theo từng thành phần.

Embedding là kỹ thuật biểu diễn văn bản thành vector số trong không gian nhiều chiều. Khi hai đoạn văn có ý nghĩa gần nhau, vector của chúng thường có độ tương đồng cao. Trong hệ thống hỏi đáp dựa trên tài liệu, embedding cho phép ánh xạ câu hỏi của người dùng và các đoạn SGK vào cùng một không gian, từ đó tìm các đoạn tài liệu gần nhất về mặt ngữ nghĩa.

Một độ đo phổ biến cho tìm kiếm ngữ nghĩa là cosine similarity:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} \quad (2.2)$$

Trong đó, $\mathbf{A}$ và $\mathbf{B}$ là hai vector biểu diễn văn bản. Giá trị càng gần 1 thì hai vector càng cùng hướng, tức nội dung văn bản càng gần nhau về mặt ngữ nghĩa.

Tìm kiếm ngữ nghĩa phù hợp với các câu hỏi được diễn đạt khác với tài liệu gốc nhưng vẫn có cùng ý nghĩa. Tuy nhiên, nó không phải lúc nào cũng xử lý tốt các thuật ngữ chính xác, tên lệnh, ký hiệu hoặc cụm từ hiếm. Vì vậy, trong các hệ thống RAG thực tế, tìm kiếm vector thường được kết hợp với các chiến lược khác như lọc metadata, tìm kiếm từ khóa hoặc reranking.

2.4 Sinh tăng cường truy xuất

Retrieval-Augmented Generation là kiến trúc kết hợp truy xuất thông tin và sinh văn bản. Thay vì để LLM trả lời trực tiếp, hệ thống trước hết truy xuất các đoạn tài liệu liên quan, sau đó đưa các đoạn này vào prompt để LLM sinh câu trả lời dựa trên tài liệu.

Một hệ thống RAG thường gồm hai pha. Pha chuẩn bị dữ liệu thực hiện làm sạch tài liệu, chia tài liệu thành chunk, tạo embedding và lưu chỉ mục cùng metadata. Pha truy vấn nhận câu hỏi người dùng, truy xuất các chunk liên quan, có thể rerank kết quả, xây dựng context và gọi LLM để sinh câu trả lời.

Trong bài toán giáo dục, RAG có ba lợi ích chính. Thứ nhất, hệ thống có thể cập nhật tri thức bằng cách cập nhật kho tài liệu thay vì huấn luyện lại mô hình. Thứ hai, câu trả lời có căn cứ hơn vì được sinh từ context cụ thể. Thứ ba, hệ thống có thể ghi lại context đã dùng để phục vụ kiểm tra, phân tích lỗi và đánh giá thực nghiệm.

Tuy nhiên, RAG không tự động bảo đảm câu trả lời đúng. Nếu chunking làm mất ngữ cảnh, retriever lấy sai đoạn, hoặc context đưa vào prompt chứa nhiều nhiễu, chất lượng câu trả lời vẫn có thể thấp. Vì vậy, đánh giá một hệ thống RAG cần tách riêng chất lượng truy xuất và chất lượng sinh câu trả lời.

2.5 Kiến thức nền tảng về truy xuất, xếp hạng lại và đánh giá

2.5.1 Lọc metadata theo phạm vi

Dữ liệu SGK có metadata tự nhiên như bộ sách, lớp, chủ đề và bài học. Khi người dùng đã nêu rõ phạm vi, ví dụ “lớp 11 Cánh Diều”, hệ thống nên giới hạn truy xuất trong phạm vi đó. Cách làm này giúp giảm nguy cơ lấy nhầm nội dung cùng thuật ngữ nhưng thuộc lớp hoặc bộ sách khác.

Tuy nhiên, lọc quá chặt cũng có thể gây mất kết quả nếu metadata thiếu hoặc câu hỏi không khớp chính xác với chủ đề. Do đó, hệ thống cần ghi nhận phạm vi yêu cầu, phạm vi thực tế đã truy xuất và các trường hợp phải mở rộng phạm vi khi không tìm được context phù hợp.

2.5.2 Viết lại truy vấn và truy xuất đa truy vấn

Câu hỏi của người dùng thường ngắn, mơ hồ hoặc phụ thuộc vào lịch sử hội thoại. Query rewriting là bước viết lại câu hỏi thành dạng rõ hơn để phục vụ truy xuất. Multi-query retrieval mở rộng ý tưởng này bằng cách sinh nhiều truy vấn phụ, sau đó truy xuất và hợp nhất kết quả.

Trong đồ án, LLM có thể được dùng để hỗ trợ tạo truy vấn giàu ngữ nghĩa hơn. Tuy nhiên, các truy vấn này chỉ phục vụ retrieval, không được xem là câu trả lời cuối cùng. Hiệu quả của query rewriting cần được kiểm tra bằng benchmark retrieval, thông qua việc so sánh kết quả trước và sau khi viết lại truy vấn.

2.5.3 Xếp hạng lại

Retriever ban đầu thường ưu tiên tốc độ và độ phủ, nên danh sách top-k có thể chứa các đoạn chưa thật sự liên quan. Reranking là bước đánh giá lại các candidate bằng một mô hình chính xác hơn, thường là cross-encoder, để đưa các đoạn phù

hợp lên vị trí cao hơn.

Reranking có thể cải thiện chất lượng context đưa vào LLM, nhưng làm tăng chi phí tính toán và độ trễ. Vì vậy, trong thực nghiệm cần so sánh kết quả retrieval trước và sau reranking để xác định lợi ích thực tế của bước này.

2.5.4 Truy xuất thích nghi

Một chiến lược truy xuất cố định không phù hợp với mọi câu hỏi. Câu hỏi định nghĩa cần đoạn nội dung chi tiết, câu hỏi về chương trình học cần metadata, còn câu hỏi tổng quan có thể cần nhiều đoạn đại diện. Adaptive retrieval là hướng chọn chiến lược truy xuất theo loại câu hỏi, phạm vi và tín hiệu ngữ cảnh.

Trong đồ án, adaptive retrieval được xem là hướng giải pháp phù hợp vì dữ liệu SGK có cấu trúc phân cấp rõ ràng và người dùng có thể hỏi theo nhiều mức độ chi tiết khác nhau. Cơ chế này cũng tạo cơ sở để so sánh nhiều cấu hình retrieval trong chương thực nghiệm.

Đánh giá hệ thống RAG cần tách hai câu hỏi: hệ thống có truy xuất đúng context không, và LLM có sinh câu trả lời tốt từ context đó không. Nếu chỉ nhìn câu trả lời cuối cùng, rất khó xác định lỗi đến từ retriever hay từ generator.

Đối với retrieval, một kết quả được xem là đúng khi các context được truy xuất chứa thông tin cần thiết để trả lời câu hỏi. Các thước đo thường dùng gồm Hit@k, Recall@k, Precision@k, MRR và nDCG nếu có nhãn mức độ liên quan. Các thước đo này có thể tính tự động nếu benchmark có expected context, expected source hoặc expected keywords đủ rõ.

Đối với generation, việc đánh giá khó tự động hóa hơn vì câu trả lời có thể đúng theo nhiều cách diễn đạt khác nhau. Trong phạm vi đồ án, chất lượng generation nên được đánh giá thủ công bằng rubric. Các tiêu chí chính gồm độ đúng nội dung, độ đầy đủ, mức độ bám context, độ rõ ràng và khả năng tránh hallucination. LLM có thể được dùng để sinh benchmark ban đầu, nhưng không được dùng làm bộ chấm chính cho generation trong giai đoạn này để tránh tăng chi phí và tránh trùng vai trò với node validate trong pipeline.

2.6 Kết chương

Chương này đã làm rõ phạm vi nghiên cứu của đồ án, bao gồm ngữ cảnh dữ liệu SGK, đối tượng sử dụng và bản chất của bài toán hỏi đáp dựa trên tài liệu. Bên cạnh đó, chương cũng đã khảo sát các hướng tiếp cận liên quan và chỉ ra rằng các giải pháp dựa trên RAG phù hợp hơn với yêu cầu bám sát SGK so với việc dùng trực tiếp mô hình ngôn ngữ lớn.

Hai nhóm kiến thức nền được trình bày trong chương, gồm RAG cho hỏi đáp

dựa trên tài liệu và các kỹ thuật retrieval, reranking cùng phương pháp đánh giá, tạo cơ sở trực tiếp cho việc thiết kế giải pháp ở chương tiếp theo. Trên nền tảng đó, Chương 3 sẽ trình bày chi tiết kiến trúc đề xuất, vai trò của từng node và phương pháp đánh giá các thành phần trọng tâm của hệ thống.

CHƯƠNG 3. THIẾT KẾ GIẢI PHÁP VÀ PHƯƠNG PHÁP ĐÁNH GIÁ

Chương 2 đã trình bày cơ sở lý thuyết, khảo sát các hướng tiếp cận và xác định RAG là hướng phù hợp cho bài toán hỏi đáp trên SGK. Trên cơ sở đó, chương này trình bày thiết kế chi tiết của giải pháp được đề xuất. Trọng tâm của chương là mô tả pipeline xử lý dữ liệu, pipeline xử lý câu hỏi, vai trò của từng node và phương pháp đánh giá các node quan trọng.

Khác với tài liệu thiết kế phần mềm thuần túy, chương này không đi sâu vào mô tả toàn bộ package hoặc giao diện web. Các thành phần phần mềm chỉ được trình bày ở mức cần thiết để làm rõ cách hệ thống hiện thực hóa giải pháp nghiên cứu. Hai node được đánh giá sâu trong đồ án là retrieval và generation. Các node còn lại như intent routing, context building, validation, session và trace được mô tả để làm rõ luồng xử lý và kiểm thử chức năng.

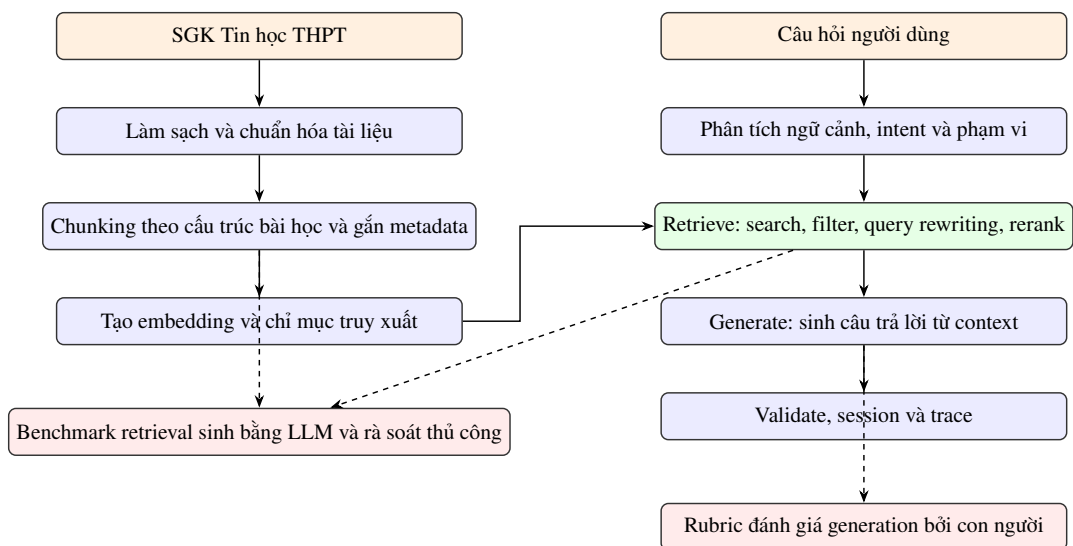
3.1 Mục tiêu thiết kế

Mục tiêu của giải pháp là xây dựng một trợ lý giáo dục có khả năng trả lời câu hỏi và hỗ trợ một số tác vụ học tập dựa trên nội dung SGK Tin học THPT. Câu trả lời của hệ thống cần bám sát tài liệu, phù hợp với phạm vi sách/lớp/chủ đề, và có thể phân tích được nguyên nhân lỗi khi kết quả chưa đạt.

Từ mục tiêu đó, thiết kế hệ thống cần thỏa mãn các yêu cầu sau. Thứ nhất, dữ liệu SGK phải được xử lý thành các đơn vị truy xuất có metadata rõ ràng. Thứ hai, hệ thống phải truy xuất được context liên quan trước khi gọi LLM. Thứ ba, pipeline cần hỗ trợ thử nghiệm nhiều chiến lược search để so sánh chất lượng retrieval. Thứ tư, câu trả lời sinh ra cần được đánh giá bằng con người theo rubric, thay vì chỉ dựa vào một node validate tự động.

3.2 Kiến trúc tổng thể của giải pháp

Giải pháp được tổ chức thành hai pipeline chính: pipeline chuẩn bị dữ liệu và pipeline xử lý truy vấn. Pipeline chuẩn bị dữ liệu chạy trước, có nhiệm vụ làm sạch tài liệu, chia chunk, tạo embedding và lưu metadata. Pipeline xử lý truy vấn chạy khi người dùng gửi câu hỏi, có nhiệm vụ phân tích ngữ cảnh, truy xuất context, rerank nếu cần và sinh câu trả lời.



Hình 3.1: Tổng quan pipeline dữ liệu, pipeline truy vấn và hai luồng đánh giá

Hình 3.1 cho thấy thiết kế không đặt trọng tâm vào giao diện ứng dụng, mà đặt trọng tâm vào mối quan hệ giữa dữ liệu, retrieval, generation và evaluation. Đây là cơ sở để Chương 4 triển khai kiểm thử và trình bày kết quả thực nghiệm.

3.3 Thiết kế dữ liệu đầu vào

Dữ liệu đầu vào của hệ thống là nội dung SGK Tin học THPT thuộc các lớp 10, 11 và 12 của hai bộ Cánh Diều và Kết Nối Tri Thức. Mỗi tài liệu có cấu trúc phân cấp gồm bộ sách, lớp, chủ đề, bài học và các mục con. Vì vậy, metadata không chỉ là thông tin phụ trợ mà là thành phần quan trọng để giới hạn phạm vi retrieval.

Mỗi chunk sau khi xử lý cần có các nhóm thông tin sau:

Bảng 3.1: Các trường metadata chính của chunk

Trường	Ý nghĩa
chunk_id	Định danh duy nhất của chunk, dùng để trace và đánh giá retrieval.
book	Bộ sách, ví dụ Cánh Diều hoặc Kết Nối Tri Thức.
grade	Lớp học tương ứng, gồm 10, 11 hoặc 12.
topic	Chủ đề hoặc mạch nội dung trong chương trình.
lesson	Tên bài học chứa chunk.
section	Mục nhỏ hoặc heading trong bài học nếu có.
text	Nội dung văn bản được dùng cho retrieval và generation.
source	Nguồn dữ liệu hoặc file gốc để kiểm tra lại khi cần.

Việc lưu metadata cho phép hệ thống thực hiện filter theo phạm vi. Ví dụ, khi người dùng hỏi về bài học lớp 11 Cánh Diều, retriever có thể ưu tiên hoặc giới hạn kết quả trong đúng lớp và bộ sách này. Điều này đặc biệt quan trọng vì nhiều thuật

ngữ Tin học xuất hiện ở nhiều lớp khác nhau.

3.4 Thiết kế tiền xử lý và chunking

Tiền xử lý tài liệu gồm các bước đọc dữ liệu nguồn, chuẩn hóa ký tự, loại bỏ đoạn rỗng hoặc nhiễu, nhận diện heading và gắn metadata. Mục tiêu của bước này là đưa tài liệu về dạng ổn định để chunking và retrieval có thể hoạt động nhất quán.

Chunking được thiết kế theo hướng ưu tiên cấu trúc bài học thay vì chỉ cắt theo độ dài cố định. Với dữ liệu SGK, mỗi heading hoặc tiểu mục thường tương ứng với một đơn vị kiến thức có ý nghĩa sư phạm. Nếu cắt ngang một định nghĩa hoặc ví dụ, context truy xuất có thể thiếu thông tin cần thiết để LLM trả lời đúng.

Tuy nhiên, chunk quá dài cũng làm retrieval kém chính xác vì chứa nhiều nội dung phụ. Vì vậy, sau khi chia theo cấu trúc, hệ thống vẫn cần kiểm soát độ dài chunk và có thể áp dụng overlap khi cần. Các tham số cần báo cáo trong thực nghiệm gồm kích thước chunk, overlap, số lượng chunk, độ dài trung bình và phân phối độ dài.

3.5 Thiết kế embedding và indexing

Sau khi có danh sách chunk, hệ thống tạo embedding cho từng chunk và lưu cùng metadata. Embedding giúp hệ thống thực hiện tìm kiếm ngữ nghĩa, trong khi metadata giúp lọc phạm vi. Hai nhóm thông tin này bổ sung cho nhau: embedding giúp tìm đoạn gần nghĩa với câu hỏi, còn metadata giúp tránh lấy nhầm đoạn thuộc phạm vi khác.

Input của node embedding là nội dung chunk. Output là vector biểu diễn và payload metadata. Trong triển khai, mô hình embedding tiếng Việt được ưu tiên vì dữ liệu SGK và câu hỏi của người dùng đều ở tiếng Việt. Việc chọn embedding tiếng Việt giúp giảm rủi ro biểu diễn sai các thuật ngữ hoặc cấu trúc ngôn ngữ đặc thù.

Index cần hỗ trợ truy xuất top-k theo độ tương đồng vector và trả về metadata đi kèm. Khi dữ liệu thay đổi, hệ thống cần tái tạo hoặc cập nhật index để đảm bảo chunk, embedding và metadata đồng bộ. Đây là điều kiện cần để benchmark retrieval có thể tính metric chính xác.

3.6 Thiết kế phân tích ngữ cảnh, intent và phạm vi

Trước khi truy xuất, hệ thống cần làm rõ câu hỏi người dùng. Một câu hỏi có thể thiếu chủ đề, phụ thuộc vào lịch sử hội thoại hoặc chứa nhiều ý định trong cùng một lượt nhập. Nếu đưa trực tiếp câu hỏi như vậy vào retriever, kết quả có thể thiếu hoặc sai phạm vi.

Node phân tích ngữ cảnh nhận câu hỏi hiện tại, thông tin người dùng, phạm vi chọn từ giao diện và session trước đó. Output của node này gồm câu hỏi đã được làm giàu, danh sách truy vấn phục vụ RAG nếu có, thông tin intent, phạm vi sách/lớp/chủ đề và các debug step. Khi câu hỏi phụ thuộc vào lịch sử, hệ thống có thể dùng thông tin phiên trước để bổ sung ngữ cảnh.

Intent routing xác định yêu cầu thuộc nhóm hỏi đáp, giải thích, sinh quiz, chấm đáp án, sinh slide, sinh giáo án hoặc fallback. Action planning ánh xạ intent thành hành động cụ thể. Trong thiết kế này, intent routing và action planning giúp đưa request vào đúng service, nhưng không phải node đánh giá định lượng chính của đồ án.

3.7 Thiết kế retrieval service

Retrieval service là thành phần trọng tâm của đồ án. Node này nhận câu hỏi hoặc danh sách truy vấn, áp dụng các cấu hình search khác nhau và trả về danh sách context candidate. Chất lượng của node retrieval ảnh hưởng trực tiếp đến chất lượng generation phía sau.

3.7.1 Vector search cơ bản

Vector search cơ bản chuyển câu hỏi thành embedding, sau đó tìm các chunk có vector gần nhất. Đây là baseline quan trọng vì hầu hết các cấu hình nâng cao đều được so sánh với nó. Output của vector search gồm danh sách chunk, điểm tương đồng và metadata.

3.7.2 Vector search có filter phạm vi

Khi người dùng cung cấp sách, lớp hoặc chủ đề, hệ thống có thể áp dụng filter metadata trước hoặc trong quá trình search. Cấu hình này giúp giảm nhiễu do các chunk thuộc lớp hoặc bộ sách khác. Tuy nhiên, nếu filter quá chặt, hệ thống có thể bỏ sót kết quả liên quan. Vì vậy, cần đánh giá riêng hiệu quả của filter scope.

3.7.3 Search với query rewriting

Query rewriting dùng LLM để viết lại câu hỏi hoặc sinh truy vấn phụ giàu ngữ nghĩa hơn. Ví dụ, câu hỏi ngắn có thể được mở rộng bằng chủ đề, lớp hoặc thuật ngữ liên quan. Các truy vấn này được dùng cho retrieval, không dùng để trả lời trực tiếp.

Đánh giá node này cần so sánh retrieval trước và sau query rewriting. Nếu rewritten query làm tăng Hit@k hoặc MRR, bước này có giá trị thực nghiệm. Nếu kết quả không cải thiện, hệ thống cần cân nhắc chi phí gọi LLM so với lợi ích đạt được.

3.7.4 Multi-query retrieval và merge kết quả

Multi-query retrieval chạy nhiều truy vấn trên cùng một câu hỏi, sau đó hợp nhất kết quả. Cách làm này tăng độ phủ nhưng có thể đưa thêm nhiều vào danh sách candidate. Vì vậy, bước merge cần loại trùng chunk, giữ metadata và có cách xếp hạng lại kết quả hợp nhất.

3.7.5 Reranking

Reranking nhận query và danh sách candidate chunks, sau đó sắp xếp lại theo mức độ liên quan. Trong pipeline RAG, reranking thường chỉ áp dụng trên số lượng candidate nhỏ để cân bằng chất lượng và độ trễ.

Trong thiết kế đánh giá, reranking được kiểm tra bằng cách so sánh các metric retrieval trước và sau khi rerank. Nếu chunk đúng được đưa lên vị trí cao hơn, MRR và nDCG sẽ cải thiện. Đây là cơ sở để quyết định có nên giữ reranking trong pipeline mặc định hay không.

3.7.6 Adaptive retrieval

Adaptive retrieval chọn chiến lược truy xuất theo loại câu hỏi và phạm vi. Ví dụ, câu hỏi về danh sách bài học có thể ưu tiên metadata, trong khi câu hỏi định nghĩa cần chunk nội dung chi tiết. Thành phần này giúp hệ thống không dùng một cấu hình search cố định cho mọi trường hợp.

Trong thực nghiệm, adaptive retrieval chỉ nên được xem là một cấu hình riêng nếu nó có chiến lược khác rõ ràng so với vector search thường. Nếu adaptive retrieval chỉ bao bọc lại vector search mà không thay đổi logic, không nên trình bày như một đóng góp độc lập.

3.8 Thiết kế context building và generation

Sau retrieval, hệ thống cần biến danh sách chunk thành context đưa vào prompt. Context building có nhiệm vụ loại trùng, sắp xếp thứ tự, giới hạn độ dài và giữ lại thông tin nguồn. Nếu context quá dài, prompt có thể chứa nhiều nhiễu. Nếu context quá ngắn, LLM có thể thiếu thông tin để trả lời.

Node generation nhận câu hỏi, context, prompt template và metadata. Output là câu trả lời cuối cùng. Prompt cần yêu cầu mô hình trả lời dựa trên context, không bịa khi context không đủ, diễn giải phù hợp với học sinh THPT và giữ đúng thuật ngữ Tin học.

Node validate nếu có được xem là bước kiểm soát nội bộ của pipeline. Node này có thể phát hiện một số lỗi format hoặc cảnh báo câu trả lời thiếu căn cứ, nhưng không thay thế đánh giá thủ công của con người đối với chất lượng generation. Đây là ranh giới quan trọng để tránh đánh đồng kiểm tra runtime với kết quả nghiên

cứu.

3.9 Thiết kế benchmark retrieval

Để đánh giá retrieval, đồ án cần một file benchmark gồm các câu hỏi và thông tin kỳ vọng. Benchmark được sinh bằng LLM từ nội dung SGK hoặc các chunk đã xử lý, sau đó được rà soát thủ công để bảo đảm câu hỏi có đáp án trong tài liệu và metadata không bị lệch.

Quy trình sinh benchmark gồm năm bước. Đầu tiên, chọn phạm vi dữ liệu theo sách, lớp, chủ đề hoặc bài học. Tiếp theo, đưa nội dung chunk hoặc bài học vào prompt để LLM sinh câu hỏi. Sau đó, yêu cầu LLM trả về câu hỏi, đáp án kỳ vọng, từ khóa kỳ vọng, metadata nguồn và loại câu hỏi. Bước thứ tư là lọc bỏ câu hỏi trùng, quá mơ hồ hoặc không trả lời được từ tài liệu. Cuối cùng, người thực hiện rà soát benchmark trước khi dùng để tính metric.

Mỗi mẫu benchmark nên có các trường sau:

Bảng 3.2: Cấu trúc đề xuất cho một mẫu retrieval benchmark

Trường	Ý nghĩa
id	Mã câu hỏi benchmark.
question	Câu hỏi dùng để chạy retrieval.
expected_answer	Đáp án tham chiếu ngắn để kiểm tra nội dung.
expected_book	Bộ sách kỳ vọng.
expected_grade	Lớp kỳ vọng.
expected_topic	Chủ đề kỳ vọng.
expected_lesson	Bài học hoặc mục chứa đáp án.
chunk_ids	Danh sách chunk kỳ vọng nếu đã có nhãn ổn định.
keywords	Từ khóa cần xuất hiện trong context đúng.
question_type	Loại câu hỏi: định nghĩa, liệt kê, so sánh, quy trình, vận dụng hoặc ngoài phạm vi.
difficulty	Mức độ khó của câu hỏi.

Khi chưa có expected chunk ổn định, benchmark có thể dùng kết hợp expected metadata và expected keywords. Tuy nhiên, để tính metric chính xác hơn, đồ án nên dần gắn expected chunk hoặc expected source cho các câu hỏi quan trọng.

3.10 Thiết kế đánh giá retrieval

Retrieval được xem là đúng khi danh sách context trả về chứa thông tin cần thiết để trả lời câu hỏi. Tùy mức độ gắn nhãn của benchmark, một context có thể được xem là liên quan nếu nó có chunk id nằm trong expected chunk, hoặc có metadata đúng và chứa các expected keywords.

Các metric chính gồm:

- **Hit@k**: bằng 1 nếu trong top-k có ít nhất một context liên quan.
- **Recall@k**: tỷ lệ context liên quan được tìm thấy trong top-k so với tập context kỳ vọng.
- **Precision@k**: tỷ lệ context trong top-k là context liên quan.
- **MRR**: nghịch đảo vị trí của context liên quan đầu tiên.
- **nDCG**: đánh giá chất lượng thứ hạng khi có nhãn relevance nhiều mức.

Các cấu hình search cần so sánh gồm vector search cơ bản, vector search có filter scope, vector search với rewritten query, multi-query retrieval, vector search kết hợp rerank và adaptive retrieval. Kết quả cần được phân tích theo loại câu hỏi, lớp, bộ sách, chủ đề và nhóm lỗi thường gặp.

3.11 Thiết kế đánh giá generation

Generation được đánh giá dựa trên câu trả lời cuối cùng mà LLM sinh ra từ context. Khác với retrieval, đánh giá generation không nên chỉ dựa vào metric tự động vì một câu trả lời đúng có thể có nhiều cách diễn đạt. Trong phạm vi đồ án, generation được chấm bởi con người theo rubric.

Rubric đề xuất gồm năm tiêu chí: correctness, faithfulness, completeness, clarity và hallucination. Correctness đánh giá câu trả lời có đúng nội dung SGK không. Faithfulness đánh giá câu trả lời có bám context truy xuất không. Completeness đánh giá mức độ đầy đủ so với expected answer. Clarity đánh giá độ rõ ràng và phù hợp với học sinh THPT. Hallucination ghi nhận việc câu trả lời có thêm thông tin không có trong context hay không.

Bảng 3.3: Rubric chấm generation theo thang 1-5

Điểm	Mô tả
5	Câu trả lời đúng, đủ, bám context rõ ràng và diễn đạt phù hợp.
4	Câu trả lời đúng ý chính, bám context, nhưng thiếu một vài chi tiết nhỏ.
3	Câu trả lời đúng một phần hoặc thiếu căn cứ ở một số ý.
2	Câu trả lời thiếu nhiều thông tin, lệch context hoặc có suy diễn đáng kể.
1	Câu trả lời sai, bịa thông tin hoặc không trả lời đúng câu hỏi.

Khi chấm, người đánh giá cần xem câu hỏi, expected answer, retrieved context và generated answer. Nếu điểm thấp, cần ghi lại lỗi chính để phân biệt lỗi do retrieval hay do generation. Node validate bằng LLM nếu có trong pipeline chỉ được dùng như thông tin tham khảo, không dùng làm ground truth chấm generation.

3.12 Thiết kế logging, session và trace

Logging và trace giúp hệ thống tái lập lỗi và phân tích kết quả thực nghiệm. Mỗi request cần lưu câu hỏi gốc, câu hỏi đã làm giàu, intent, action plan, phạm vi truy xuất, danh sách chunk lấy được, điểm retrieval hoặc rerank, prompt metadata, câu trả lời sinh ra và thông tin đánh giá nếu có.

Session lưu trạng thái hội thoại và các tác vụ tương tác như quiz hoặc duyệt dàn ý slide. Trong nghiên cứu retrieval/generation, session không phải node đánh giá chính, nhưng nó giúp kiểm tra các câu hỏi phụ thuộc ngữ cảnh và hỗ trợ phân tích các request nối tiếp.

3.13 Kết chương

Chương này đã trình bày thiết kế giải pháp theo hướng nghiên cứu. Hệ thống được chia thành pipeline chuẩn bị dữ liệu và pipeline xử lý truy vấn. Các node như chunking, embedding, context analysis, retrieval, reranking, generation, validation và trace được mô tả theo vai trò trong pipeline.

Trọng tâm đánh giá của đồ án là node retrieval và node generation. Retrieval được đánh giá định lượng bằng benchmark sinh từ SGK với sự hỗ trợ của LLM và được rà soát thủ công. Generation được đánh giá bằng con người theo rubric để tránh phụ thuộc vào LLM-as-judge và tránh tăng chi phí. Chương tiếp theo sẽ trình bày cài đặt hệ thống, kiểm thử và kết quả thực nghiệm tương ứng với thiết kế này.

CHƯƠNG 4. CÀI ĐẶT, KIỂM THỬ VÀ KẾT QUẢ THỰC NGHIỆM

Chương 3 đã trình bày thiết kế giải pháp và phương pháp đánh giá các node trọng tâm trong pipeline RAG. Trên cơ sở đó, Chương 4 mô tả hiện trạng cài đặt của hệ thống, các bước kiểm thử chức năng và các kết quả thực nghiệm đã thu được. Nội dung chương tập trung vào môi trường thực nghiệm, dữ liệu SGK, kiến trúc xử lý yêu cầu, cơ chế truy xuất tri thức, các pipeline nghiệp vụ và kết quả đánh giá retrieval/generation.

Mục tiêu của chương này là làm rõ hệ thống đã được hiện thực hóa như thế nào so với thiết kế ở Chương 3. Các sơ đồ trong chương được trình bày theo từng lớp xử lý để thể hiện vai trò của mỗi khối trong vòng đời yêu cầu, đồng thời phân biệt rõ bộ điều phối cấp hệ thống, các dịch vụ nghiệp vụ, quy trình đa tác tử nội bộ và lớp truy xuất tri thức.

4.1 Môi trường và dữ liệu thực nghiệm

4.1.1 Môi trường phát triển

Hệ thống được xây dựng chủ yếu bằng ngôn ngữ lập trình Python, kết hợp với các thư viện xử lý số như Numpy và Pandas để thao tác trên ma trận vector nhúng. Phần điều phối tổng thể được thiết kế theo hướng bộ điều khiển luồng mỏng; các quyết định định tuyến chính được giữ ở tầng mã nguồn để giảm độ trễ, hỗ trợ gỡ lỗi và thuận lợi cho việc theo vết. LangGraph không được sử dụng như nền tảng bao trùm toàn bộ hệ thống, mà được dùng tập trung cho bộ điều phối sinh học liệu dài, phục vụ hai tác vụ sinh slide và giáo án.

Việc sinh ngôn ngữ tự nhiên được thực hiện thông qua API Google Gemini với mô hình `gemini-2.5-flash-lite` qua thư viện `google-genai`. Đối với truy xuất ngữ nghĩa, hệ thống sử dụng mô hình nhúng `dangvantuan/vietnamese-document-embedding`, tạo vector 768 chiều cho mỗi khối dữ liệu. Giai đoạn tái xếp hạng ở cuối quy trình truy xuất được đảm nhiệm bởi mô hình Cross-Encoder `AITeamVN/Vietnamese_Reranker`. Ngoài truy xuất ngữ nghĩa, hệ thống tự xây dựng thành phần tìm kiếm từ khóa bằng Custom BM25 và hợp nhất kết quả với semantic search thông qua Reciprocal Rank Fusion (RRF).

4.1.2 Dữ liệu thực nghiệm

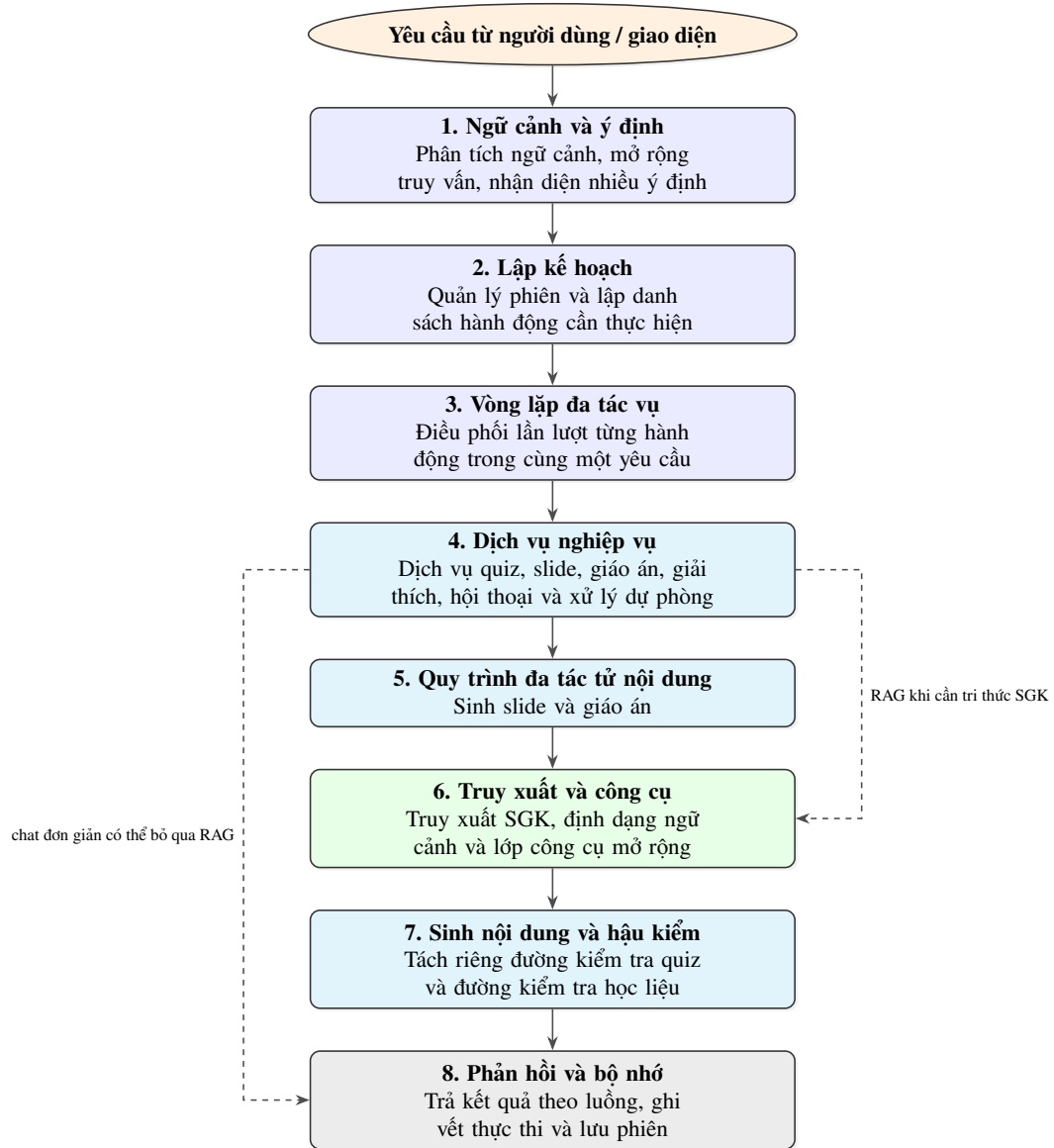
Nguồn dữ liệu gốc được khai thác từ toàn bộ tài liệu Sách Giáo Khoa Tin học lớp 10, 11 và 12, thuộc hai bộ sách Cánh Diều và Kết Nối Tri Thức. Qua quá trình tiền xử lý, văn bản được làm sạch và dán nhãn phân cấp từ chủ đề, bài học đến tiểu mục. Mỗi bài học trong một cuốn sách được coi là một tài liệu nguồn, sau đó được

chia nhỏ theo các heading bên trong bài để giữ nguyên ranh giới của từng đơn vị kiến thức. Tổng cộng, dữ liệu được phân rã thành 2348 khối dữ liệu, mỗi khối có thông tin mô tả như bộ sách, lớp, chủ đề, tên bài, kiểu nội dung và cấp phân cấp. Cấu trúc này phục vụ trực tiếp cho truy xuất phân cấp (Hierarchical RAG) và các tác vụ sinh học liệu cần bao phủ toàn bộ bài học.

4.2 Tổng quan kiến trúc toàn bộ hệ thống

Kiến trúc hiện tại của hệ thống được tổ chức quanh một trạng thái yêu cầu trung tâm. Mỗi yêu cầu người dùng đi qua các bước phân tích ngữ cảnh, nhận diện ý định, lập kế hoạch hành động, thực thi qua các dịch vụ nghiệp vụ, truy xuất tri thức khi cần, sinh kết quả, hậu kiểm chất lượng, rồi lưu trạng thái phiên và lịch sử thực thi. Thiết kế này tách rõ ba nhóm trách nhiệm: điều phối luồng chạy, xử lý nghiệp vụ, và cung cấp năng lực truy xuất/công cụ.

Sơ đồ tổng quan tại Hình 4.1 thể hiện vòng đời xử lý chính của một yêu cầu trong hệ thống.



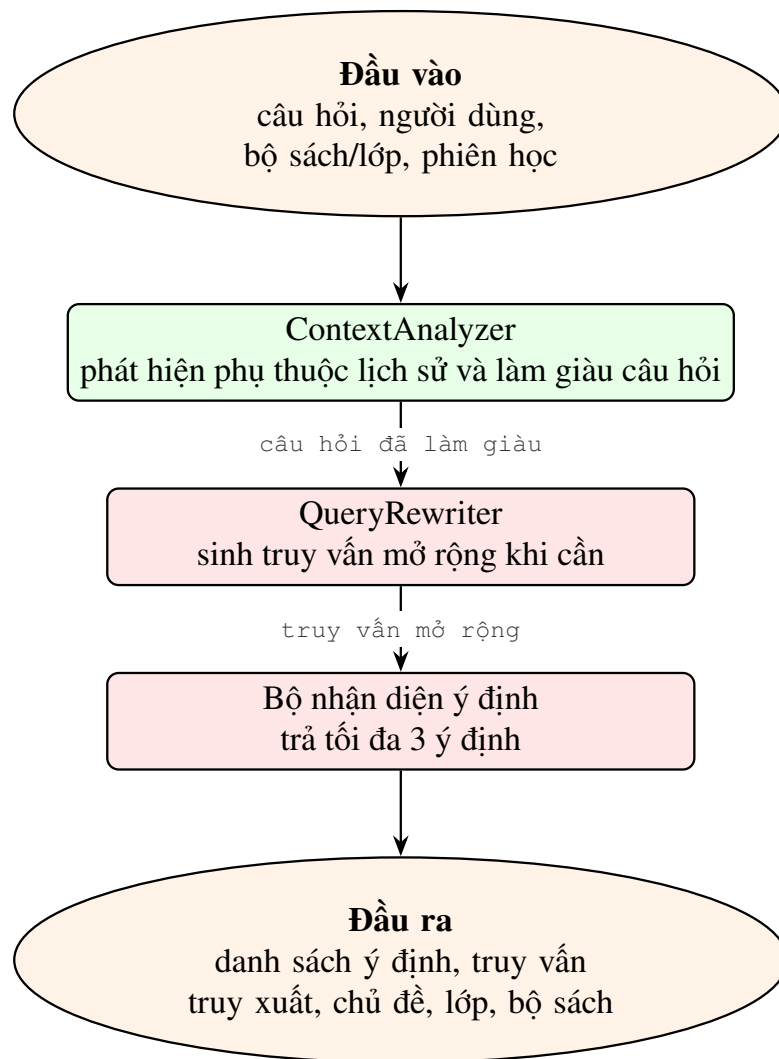
Hình 4.1: Sơ đồ tổng quan kiến trúc hệ thống theo vòng đời xử lý yêu cầu

Trong kiến trúc này, bộ điều phối không đóng vai một agent tự trị toàn phần. Nó chỉ giữ trạng thái xử lý của yêu cầu, gọi mô hình ngôn ngữ ở các vị trí thật sự cần suy luận ngôn ngữ, và sử dụng mã nguồn xác định cho các bước lập kế hoạch, định tuyến, lưu phiên và ghi vết. Nhờ vậy, hệ thống vừa hỗ trợ nhiều ý định trong một yêu cầu, vừa tránh việc tạo thêm nhiều lượt gọi mô hình không cần thiết ở các bước có thể quyết định bằng logic.

4.3 Thiết kế chi tiết các thành phần hệ thống

4.3.1 Thiết kế Lớp Ngữ cảnh và Ý định (Context & Intent Layer)

Đầu vào của hệ thống gồm câu hỏi gốc, thông tin người dùng, bộ sách/lớp được chọn từ giao diện, lịch sử phiên, các truy vấn phục vụ truy xuất tri thức và thông tin hỗ trợ theo vết. Lớp Ngữ cảnh và Ý định có nhiệm vụ làm rõ yêu cầu hiện tại trước khi hệ thống lập kế hoạch hành động.



Hình 4.2: Chi tiết cấu trúc và luồng xử lý tại Lớp Ngữ cảnh và Ý định

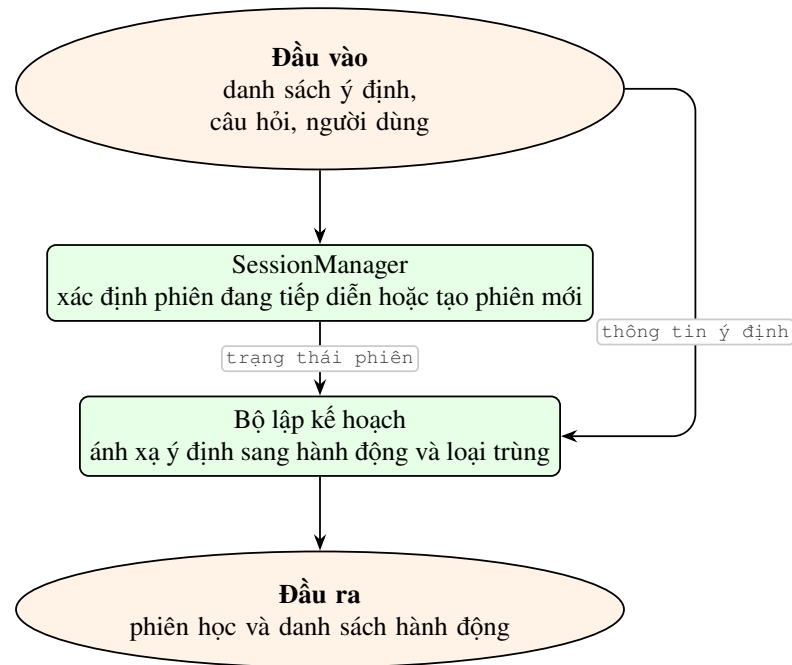
ContextAnalyzer được triển khai bằng logic mã nguồn để đánh giá câu hỏi hiện tại có phụ thuộc vào lịch sử hội thoại hay không. Khi cần, hệ thống trích xuất các lượt hội thoại gần và liên quan để bổ sung vào câu hỏi hiện tại.

QueryRewriter chỉ được gọi khi cần làm giàu truy vấn cho RAG. Thành phần này có thể tạo nhiều truy vấn ngữ nghĩa từ cùng một câu hỏi, giúp retrieval bao phủ tốt hơn những câu hỏi thiếu ngữ cảnh.

Bộ nhận diện ý định là lời gọi LLM quan trọng ở đầu quy trình. Thành phần này phân tích đồng thời nhiều ý định trong một câu hỏi và trả về danh sách ý định gồm loại yêu cầu chính, chủ đề, lớp, bộ sách và thứ tự xử lý. Giới hạn tối đa 3 ý định được dùng để tránh mở rộng trạng thái quá mức khi người dùng nhập một yêu cầu phức hợp.

4.3.2 Thiết kế Lớp Lập kế hoạch (Planning Layer)

Sau khi có danh sách ý định, hệ thống chuyển sang tầng lập kế hoạch hoàn toàn bằng luật. Tầng này quản lý phiên học và lập kế hoạch hành động; không sử dụng thêm LLM để quyết định hành động.



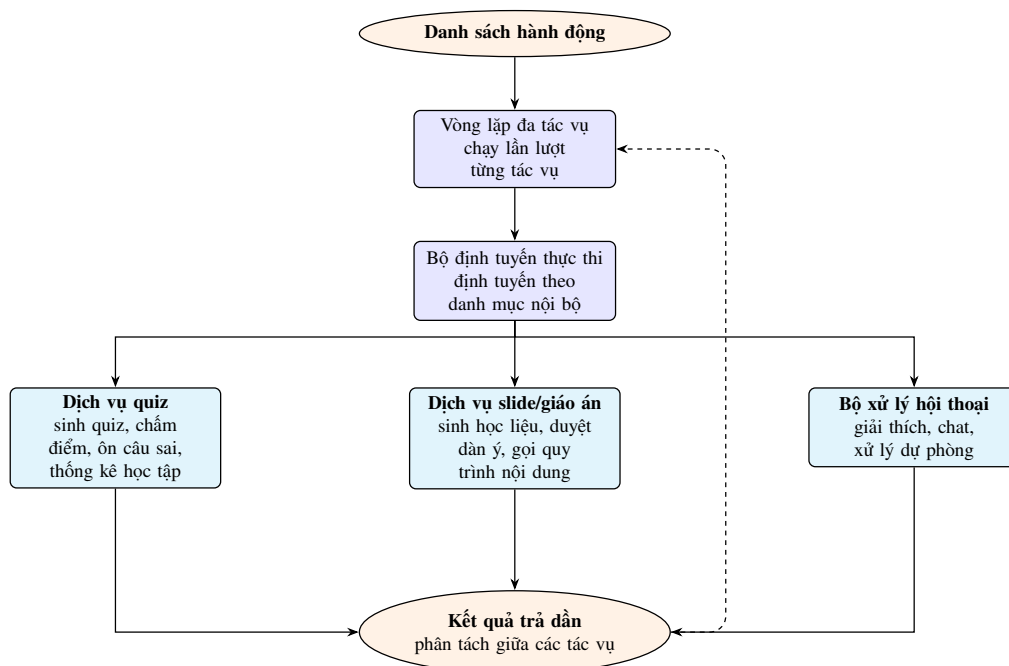
Hình 4.3: Chu trình lập kế hoạch hành động bằng rule-based logic

SessionManager kiểm tra phiên tương ứng với người dùng và chủ đề hiện tại. Thành phần này quyết định nên tiếp tục phiên đang có, khôi phục từ bộ nhớ phiên, hay tạo phiên mới.

Bộ lập kế hoạch ánh xạ mỗi ý định sang một hành động cụ thể. Các hành động chính bao gồm sinh quiz, sinh slide, sinh giáo án, chấm đáp án, ôn câu sai, thống kê tiến độ, giải thích khái niệm và chat. Khi nhiều ý định liên tiếp tạo ra cùng một hành động, bộ lập kế hoạch loại bỏ trùng lặp để tránh thực thi dư thừa.

4.3.3 Thiết kế Lớp Thực thi (Execution Layer)

Lớp Thực thi nhận danh sách hành động và xử lý theo vòng lặp đa tác vụ. Ở mỗi vòng, bộ điều phối chọn tác vụ đang chạy, sau đó chuyển quyền thực thi cho bộ định tuyến thực thi. Bộ định tuyến dùng danh mục nội bộ để gọi đúng dịch vụ hoặc bộ xử lý.



Hình 4.4: Sơ đồ vòng lặp đa tác vụ và điều phối dịch vụ nghiệp vụ

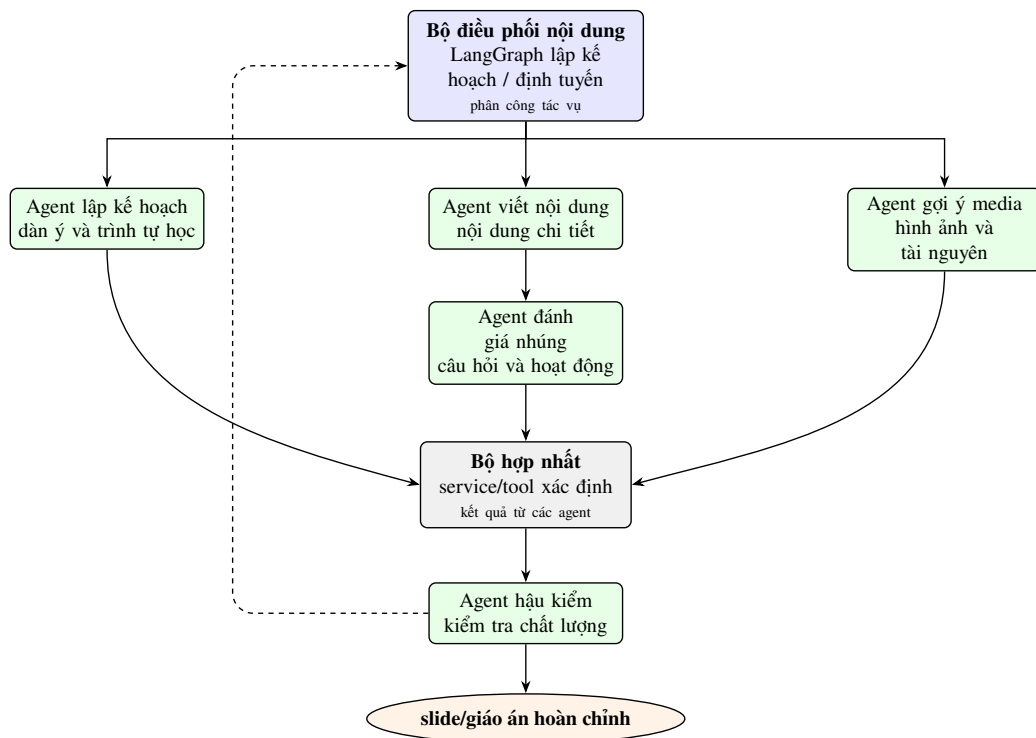
Dịch vụ quiz sở hữu toàn bộ vòng đời quiz độc lập: sinh câu hỏi, hậu kiểm chất lượng, chấm đáp án, giải thích câu hỏi, ôn câu sai, thống kê và cập nhật tiến độ học tập. Vì luồng này có nhiều trạng thái tương tác như lướt làm quiz và bản ghi từng câu hỏi, nó được giữ ở dạng dịch vụ nghiệp vụ thay vì đưa vào quy trình đa tác tử sinh học liệu.

Dịch vụ slide/giáo án là lớp mặt tiền cho cả hai tác vụ sinh slide và giáo án. Dịch vụ này truy xuất dữ liệu từ lớp tri thức, tạo đầu vào cho bộ điều phối nội dung, quản lý bước duyệt dàn ý có con người tham gia, và lưu thông tin tóm tắt để có thể tiếp tục quy trình ở lượt tương tác sau.

Các bộ xử lý giải thích, hội thoại tự do và xử lý dự phòng đảm nhiệm các nhánh hội thoại hẹp hơn. Với câu hỏi cần tri thức SGK, bộ định tuyến gọi RAG trước khi đưa ngữ cảnh vào bộ xử lý; với hội thoại tự do, hệ thống có thể bỏ qua RAG để giảm độ trễ.

4.3.4 Thiết kế Quy trình Đa tác tử cho Slide và Giáo án

Hai chức năng sinh slide và sinh giáo án có cùng bản chất nghiệp vụ: cần lấy nội dung SGK liên quan, lập trình tự sự phạm, viết nội dung chi tiết, gợi ý media, sinh hoạt động đánh giá nhúng, hợp nhất kết quả trung gian và kiểm tra chất lượng. Vì vậy hệ thống dùng chung một bộ điều phối nội dung thay vì duy trì hai bộ điều phối riêng. Khác biệt giữa hai tác vụ được biểu diễn bằng loại học liệu cần sinh: slide hoặc giáo án.



Hình 4.5: Pipeline đa tác tử nội bộ cho sinh slide và giáo án

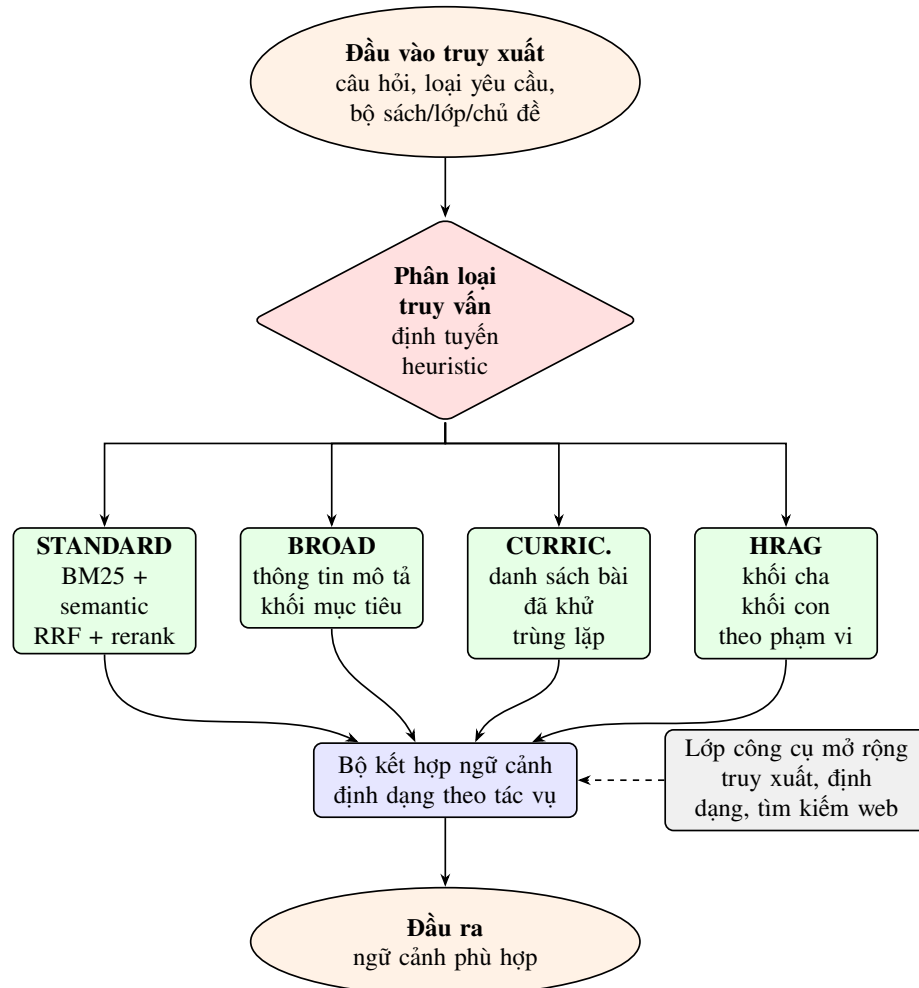
Giao tiếp giữa bộ điều phối và các agent chuyên trách sử dụng hợp đồng A2A-lite nội bộ. Mỗi nhiệm vụ được mô tả bằng bên gửi, bên nhận, loại nhiệm vụ, mục tiêu, dữ liệu vào, ràng buộc và kết quả kỳ vọng. Agent trả về trạng thái xử lý, kết quả trung gian, mức tin cậy, cảnh báo, danh sách công cụ đã dùng, độ trễ và lỗi nếu có. Các kết quả này được lưu trong trạng thái của quy trình qua ba nhóm dữ liệu: nhiệm vụ đã giao, kết quả agent và danh mục kết quả trung gian.

Agent lập kế hoạch sơ phạm tạo dàn ý và trình tự học tập. Agent viết nội dung tạo nội dung chi tiết cho từng slide hoặc từng mục trong giáo án dựa trên dàn ý và ngữ cảnh nguồn. Agent gợi ý media hiện đề xuất thông tin hình ảnh và tài nguyên; tìm kiếm ảnh và kiểm tra URL qua công cụ ngoài được xem là điểm mở rộng, chưa mô tả như năng lực production hoàn chỉnh. Agent đánh giá những chỉ sinh câu hỏi hoặc hoạt động đánh giá trong slide/giáo án, không thay thế dịch vụ quiz độc lập. Agent hậu kiểm chất lượng kiểm tra độ đúng sự kiện, độ bao phủ, tính sơ phạm và định dạng sau khi hợp nhất.

Thành phần hợp nhất kết quả không được xem là agent. Đây là service/tool xác định, có nhiệm vụ hợp nhất dàn ý, nội dung, media và phần đánh giá thành danh sách slide/section cuối cùng. Việc tách bước hợp nhất khỏi agent giúp hệ thống tránh gán tính tự trị cho một bước không cần suy luận LLM.

4.3.5 Thiết kế Lớp Tri thức (Knowledge Layer)

Lớp Tri thức cung cấp giao diện truy xuất chính cho các thành phần nghiệp vụ. Bên trong, bộ truy xuất thích nghi sử dụng bộ phân loại truy vấn dựa trên heuristic, không gọi LLM, để chọn chiến lược truy xuất phù hợp với loại câu hỏi và phạm vi lớp/bộ sách/chủ đề.



Hình 4.6: Cấu trúc xử lý và truy xuất thông tin của Knowledge Layer

STANDARD phục vụ các truy vấn cụ thể bằng Hybrid Search: Custom BM25 và semantic search được hợp nhất qua RRF, sau đó đi qua Cross-Encoder Reranker để lấy nhóm khối dữ liệu liên quan nhất. **BROAD** dùng tra cứu thông tin mô tả trên các khối mục tiêu, phù hợp với truy vấn tổng quan. **CURRICULUM** trả danh sách bài/chủ đề đã khử trùng lặp, không cần tìm kiếm vector. **HIERARCHICAL** trước tiên tìm các khối cha ở cấp cao, sau đó tìm tiếp các khối con trong cùng bài/chủ đề.

Nếu **BROAD** hoặc **HIERARCHICAL** không tìm đủ ngữ cảnh, hệ thống có thể chuyển sang **STANDARD** để bảo vệ độ phủ. Sau truy xuất, bộ kết hợp ngữ cảnh định dạng thông tin theo loại tác vụ: các tác vụ hỏi đáp hoặc quiz ưu tiên thứ tự điểm liên quan, còn slide và giáo án nhóm ngữ cảnh theo chủ đề/bài học để giữ

mạch sư phạm.

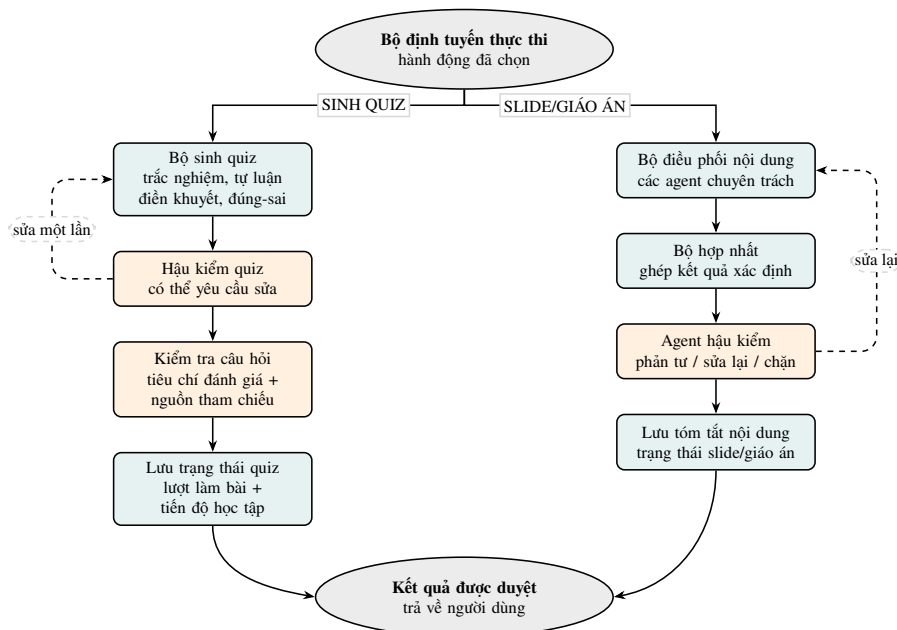
Kiến trúc MCP trong project hiện ở dạng lớp công cụ chạy trong tiến trình, gồm thành phần quản lý client, server và danh mục công cụ. Các công cụ chính phục vụ truy xuất tri thức và định dạng nội dung. Chức năng tìm kiếm web được xem là điểm mở rộng cho media/resource search, không phải kết quả thực nghiệm chính trong chương này.

4.3.6 Thiết kế Lớp Sinh phản hồi và Hậu kiểm (Generation & Validation)

Tầng sinh và hậu kiểm hiện không còn là một bộ kiểm tra chung cho mọi nhánh. Hệ thống tách rõ hai đường kiểm soát chất lượng: quiz độc lập và quy trình sinh slide/giáo án.

Ở nhánh quiz độc lập, yêu cầu của người dùng thường tạo ra một đối tượng có vòng đời tương tác dài. Câu hỏi sau khi sinh không chỉ được trả về ngay, mà còn được lưu để phục vụ chấm điểm, giải thích đáp án, ôn câu sai và thống kê tiến độ. Vì vậy nhánh này cần kiểm soát chặt chất lượng từng câu hỏi trước khi ghi vào trạng thái phiên.

Ở nhánh slide và giáo án, đầu ra là một sản phẩm tổng hợp gồm nhiều phần nội dung. Các agent chuyên trách tạo từng nhóm nội dung riêng, sau đó bước hợp nhất ghép kết quả theo một quy tắc xác định trước khi chuyển sang hậu kiểm. Agent hậu kiểm chất lượng không thay thế các agent sinh nội dung, mà chỉ đánh giá kết quả đã hợp nhất để quyết định chấp nhận, yêu cầu sửa lại hoặc chặn đầu ra lỗi nghiêm trọng.



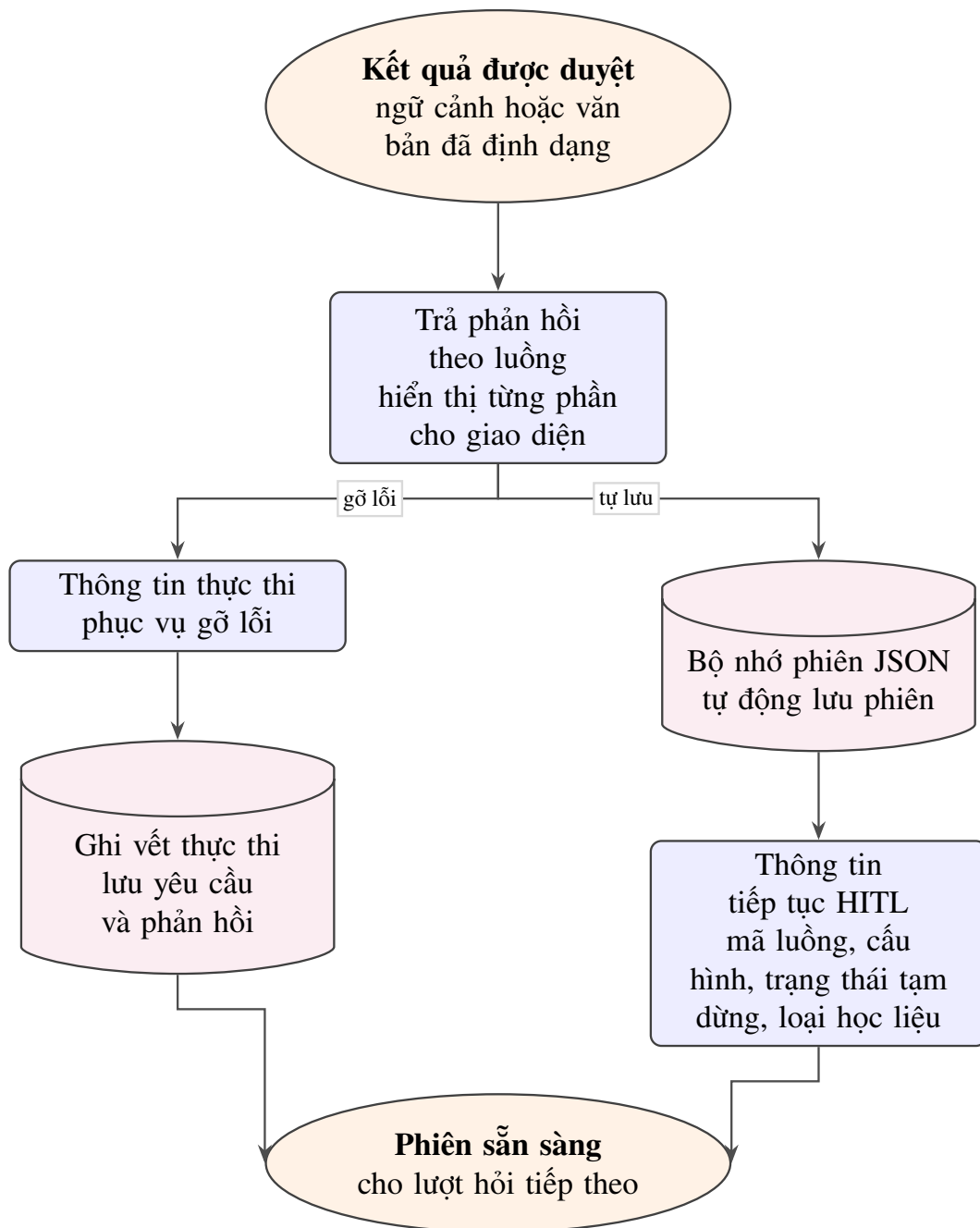
Hình 4.7: Hai đường sinh và hậu kiểm chất lượng trong hệ thống

Với **quiz độc lập**, dịch vụ quiz lấy ngữ cảnh từ RAG, gọi bộ sinh tương ứng với loại câu hỏi, chạy hậu kiểm chất lượng, có thể yêu cầu sửa một lần, sau đó kiểm tra câu hỏi theo tiêu chí và nguồn tham chiếu. Chỉ những câu hỏi được phê duyệt mới được lưu vào trạng thái lượt làm quiz. Các bước chấm điểm, ôn câu sai và thống kê tiếp tục đọc từ trạng thái quiz này.

Với **slide và giáo án**, dịch vụ slide gọi bộ điều phối nội dung. Sau khi các agent chuyên trách trả kết quả, bước hợp nhất ghép các phần thành đầu ra cuối. Agent hậu kiểm đánh giá độ đúng sự kiện, độ bao phủ, tính sư phạm và định dạng. Nếu kết quả chưa đạt, bộ điều phối có thể định tuyến lại agent chịu trách nhiệm để sửa; nếu lỗi nghiêm trọng, đầu ra bị chặn thay vì trả về người dùng.

4.3.7 Thiết kế Trả phản hồi, Lưu phiên và Ghi vết

Chặng cuối của quy trình chịu trách nhiệm trả kết quả theo luồng, ghi vết thực thi và lưu trạng thái phiên. Thiết kế này giúp người dùng nhận phản hồi sớm trong khi hệ thống vẫn giữ được lịch sử đủ để tiếp tục các tác vụ tương tác như chấm bài, ôn câu sai hoặc tiếp tục quy trình sinh slide/giáo án.



Hình 4.8: Khối phản hồi, ghi trace và lưu trạng thái phiên

Bộ nhớ phiên lưu dữ liệu cục bộ dưới dạng JSON, cho phép hệ thống khôi phục lịch sử hội thoại, trạng thái quiz và trạng thái slide giữa các lượt tương tác. Thành phần ghi vết lưu lại đường đi của yêu cầu, bao gồm các bước thực thi phục vụ gỡ lỗi. Riêng với quy trình slide/giáo án, khi bộ điều phối nội dung dừng ở bước duyệt dàn ý, dịch vụ slide lưu thông tin tiếp tục gồm mã luồng, cấu hình luồng, cờ tạm dừng và loại học liệu để nối lại đúng quy trình sau khi người dùng gửi phản hồi.

4.4 Đánh giá hiệu năng và Kết quả thực nghiệm

4.4.1 Phương pháp đánh giá

Hệ thống sử dụng phương pháp đánh giá định lượng dành riêng cho hệ thống Retrieval-Augmented Generation (RAG) thông qua Ragas framework. Tập dữ liệu kiểm thử gồm 246 mẫu câu hỏi và đáp án tham chiếu được trích xuất từ dữ liệu giáo khoa, tập trung vào nhiều cấp độ hành vi nhận thức từ nhận biết, thông hiểu đến vận dụng.

Quá trình đánh giá được thực hiện trên 4 độ đo cốt lõi của Ragas:

Faithfulness (Độ trung thực): Đo lường mức độ câu trả lời của mô hình bám sát vào ngữ cảnh mà hệ thống truy xuất được. Mức điểm cao chứng tỏ mô hình không bịa đặt thông tin ngoài nguồn.

Answer Relevancy (Độ phù hợp): Đánh giá xem câu trả lời có giải quyết trực tiếp truy vấn ban đầu của người dùng hay không, tránh trả lời lan man hoặc sai trọng tâm.

Context Precision (Độ chính xác ngữ cảnh): Đánh giá khả năng xếp hạng của cụm truy xuất. Điểm số cao đồng nghĩa với việc các tài liệu liên quan nhất được xếp ở các thứ hạng đầu.

Context Recall (Độ bao phủ ngữ cảnh): Đo lường khả năng hệ thống tìm ra đầy đủ các thông tin cần thiết từ cơ sở tri thức so với đáp án tham chiếu.

4.4.2 Kết quả thực nghiệm phân hệ RAG

Sau quá trình chạy tự động 246 mẫu thử trên quy trình RAG của hệ thống, kết quả điểm số đánh giá trung bình trên các độ đo như sau:

Chỉ số đánh giá (Metric)	Điểm số trung bình (Mean)
Faithfulness (Độ trung thực)	0.9757
Answer Relevancy (Độ thiết thực của câu trả lời)	0.8571
Context Precision (Độ chính xác của ngữ cảnh)	0.8555
Context Recall (Độ bao phủ của ngữ cảnh)	0.9593

Bảng 4.1: Kết quả đánh giá hệ thống theo framework Ragas

Phân tích kết quả: Faithfulness đạt 0.9757 và Context Recall đạt 0.9593. Kết quả này cho thấy chiến lược Hierarchical RAG kết hợp với chunking theo phân cấp SGK đã giúp hệ thống truy xuất được phần lớn các đơn vị kiến thức cần thiết, đồng

thời hạn chế hiện tượng sinh thông tin ngoài ngữ cảnh.

Context Precision (0.8555) và **Answer Relevancy (0.8571)** ổn định quanh mức 85%. Kết quả này phản ánh hiệu quả của việc kết hợp BM25, semantic search, RRF và Cross-Encoder Reranker trong việc đưa các khối dữ liệu liên quan lên nhóm đầu. Khi ngữ cảnh đầu vào có độ liên quan cao hơn, lời nhắc sinh câu trả lời có cơ sở tốt hơn để bám sát trọng tâm câu hỏi.

Các tầng chất lượng trong hệ thống đóng vai trò hậu kiểm theo từng loại tác vụ. Các thành phần này không thay thế RAG, mà bổ sung thêm lớp kiểm soát để giảm rủi ro trả về nội dung sai, thiếu căn cứ hoặc không đúng định dạng.

4.4.3 Đánh giá độ trễ hệ thống (Latency)

Bên cạnh chất lượng phản hồi, hệ thống còn được đo đạc về mặt thời gian thực thi nhằm đánh giá trải nghiệm người dùng thực tế. Các chỉ số thời gian được ghi nhận cho mỗi truy vấn đầy đủ như sau:

Thành phần thực thi	Thời gian trung bình (Giây)
Khối truy xuất (Retriever & Reranker)	4.579
Khối sinh câu trả lời (LLM Generator)	1.979
Tổng thời gian xử lý	6.558

Bảng 4.2: Thống kê thời gian trễ của các khối kiến trúc trong hệ thống

Tổng thời gian cho một vòng xử lý RAG đầy đủ đạt trung bình xấp xỉ 6.5 giây. Trọng số thời gian chủ yếu nằm ở khâu truy xuất và reranking do hệ thống phải thực hiện Hybrid Search, hợp nhất kết quả và tái xếp hạng bằng Cross-Encoder. Tuy nhiên, cơ chế streaming ở tầng phản hồi giúp người dùng nhận kết quả theo từng phần thay vì phải chờ toàn bộ nội dung hoàn tất mới hiển thị.

Chương này đã trình bày hiện trạng triển khai của hệ thống trợ lý giáo dục theo vòng đời xử lý một yêu cầu. Kiến trúc hiện tại sử dụng trạng thái yêu cầu trung tâm, bộ điều phối mỏng, bộ định tuyến thực thi và các dịch vụ nghiệp vụ để xử lý từng nhóm tác vụ. Quy trình sinh slide và giáo án được tách thành một nhánh đa tác tử riêng, trong khi quiz độc lập tiếp tục được quản lý bởi dịch vụ quiz để bảo toàn các chức năng chấm điểm, ôn tập và thống kê.

Các kết quả thực nghiệm cho thấy quy trình RAG đạt điểm cao ở hai khía cạnh trung thực và bao phủ ngữ cảnh. Điều này phù hợp với mục tiêu của đề án là xây dựng một hệ thống sinh phản hồi có căn cứ từ SGK thay vì chỉ dựa vào tri thức nội tại của mô hình ngôn ngữ lớn. Những kết quả và giới hạn triển khai được trình bày

trong chương này là cơ sở để Chương 5 phân tích sâu hơn các giải pháp và đóng góp kỹ thuật của đề án.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Các chương trước đã trình bày nền tảng lý thuyết và kết quả triển khai của hệ thống. Chương này tập trung phân tích các giải pháp và đóng góp nổi bật của đề án dưới góc nhìn thiết kế kỹ thuật. Trọng tâm không nằm ở việc liệt kê lại từng module, mà làm rõ các vấn đề khó đã gặp phải trong quá trình xây dựng trợ lý giáo dục dựa trên LLM và cách hệ thống giải quyết các vấn đề đó bằng kiến trúc có thể kiểm soát, dễ mở rộng và phù hợp với dữ liệu sách giáo khoa.

Các đóng góp chính gồm thiết kế điều phối request ở mức mã nguồn, cơ chế truy xuất thích nghi, nhận diện đa ý định, tách biệt pipeline quiz tương tác khỏi pipeline sinh nội dung dài, tổ chức content multi-agent cho slide và giáo án, chuẩn hóa giao tiếp nội bộ giữa các agent bằng A2A-lite, bổ sung HITL tại điểm quyết định quan trọng, và xây dựng các lớp kiểm tra chất lượng, session, trace phục vụ vận hành.

5.1 Điều phối request bằng thin orchestrator

5.1.1 Vấn đề cần giải quyết

Một hệ thống trợ lý giáo dục không chỉ nhận câu hỏi và gọi LLM để trả lời. Cùng một giao diện chat phải xử lý nhiều nhóm tác vụ khác nhau như hỏi đáp kiến thức, sinh câu hỏi luyện tập, chấm câu trả lời, ôn lại câu sai, tạo slide, tạo giáo án hoặc tiếp tục một pipeline đang chờ người dùng duyệt. Nếu đưa toàn bộ luồng xử lý vào một prompt lớn hoặc một agent tự quyết định mọi bước, hệ thống sẽ khó kiểm soát, khó gỡ lỗi và dễ gọi sai chức năng.

Vấn đề phức tạp hơn khi mỗi request cần mang theo nhiều loại trạng thái. Một câu hỏi có thể phụ thuộc lịch sử hội thoại, phạm vi sách/lớp người dùng chọn, chủ đề đã học trước đó, quiz state hoặc slide state đang chờ resume. Vì vậy, hệ thống cần một lớp điều phối rõ ràng để tách phần hiểu ngôn ngữ tự nhiên khỏi phần thực thi nghiệp vụ.

5.1.2 Giải pháp

Đề án thiết kế Orchestrator theo hướng thin controller. Orchestrator không chứa trực tiếp logic sinh câu hỏi, truy xuất tài liệu hoặc sinh slide, mà chủ yếu tạo `RequestContext`, phân tích ngữ cảnh, gọi `IntentRouter`, lập action plan và chuyển quyền thực thi cho `ExecutionDispatcher`. Cách làm này giữ Orchestrator ở vai trò điều phối, còn nghiệp vụ chi tiết được đóng gói trong các domain service hoặc handler.

`RequestContext` là state object đi xuyên qua toàn bộ vòng đời request. Nó lưu câu hỏi gốc, truy vấn đã làm giàu, danh sách query dùng cho RAG, intent,

action plan, phạm vi sách/lớp, session và debug steps. Nhờ đó, các module không cần truyền nhiều biến rời rạc hoặc phụ thuộc vào global state. Khi cần kiểm tra một lỗi, hệ thống có thể nhìn lại từng bước request đã đi qua.

Một điểm quan trọng là `ActionPlanner` hoạt động bằng rule-based logic thay vì gọi thêm LLM. LLM chỉ được dùng ở bước cần hiểu ngôn ngữ tự nhiên, còn ánh xạ từ intent sang action được giữ bằng mã nguồn tường minh. Điều này giảm độ trễ, tránh phát sinh nhánh xử lý ngoài kiểm soát và giúp việc thêm action mới có điểm mở rộng rõ ràng.

5.1.3 Đóng góp đạt được

Thiết kế này giải quyết được mâu thuẫn giữa tính linh hoạt của giao diện hội thoại và yêu cầu kiểm soát của hệ thống giáo dục. Người dùng vẫn có thể nhập yêu cầu tự nhiên, nhưng hệ thống không giao toàn bộ quyền điều phối cho LLM. Mỗi action được dispatch đến đúng service phụ trách, giúp kiến trúc dễ mở rộng hơn khi bổ sung chức năng mới.

Thin orchestrator cũng tạo nền tảng cho trace và debug. Mỗi request có request id, timestamp, debug steps và thông tin strategy RAG đã chọn. Đây là đóng góp thực tế quan trọng vì hệ thống LLM thường khó quan sát nếu chỉ nhìn đầu vào và đầu ra cuối cùng.

5.2 Truy xuất lại và Adaptive RAG

5.2.1 Vấn đề cần giải quyết

Dữ liệu sách giáo khoa có đặc điểm khác với dữ liệu web mở. Nhiều khái niệm quan trọng được trình bày ngắn, có thuật ngữ chuyên ngành, tên lệnh, tên bài học hoặc cấu trúc chương mục rõ ràng. Nếu chỉ dùng tìm kiếm ngữ nghĩa, hệ thống có thể bỏ sót các đoạn có từ khóa chính xác. Nếu chỉ dùng tìm kiếm từ khóa, hệ thống khó xử lý câu hỏi diễn đạt tự nhiên khác với văn bản trong sách.

Ngoài ra, không phải câu hỏi nào cũng cần cùng một cách truy xuất. Câu hỏi “giải thích khái niệm cơ sở dữ liệu” cần đoạn nội dung chi tiết, trong khi câu hỏi “lớp 12 có những chủ đề nào” cần tổng hợp cấu trúc chương trình. Một pipeline RAG cố định dễ hoặc lấy thiếu ngữ cảnh, hoặc tốn chi phí không cần thiết.

5.2.2 Giải pháp

Hệ thống sử dụng truy xuất lại gồm tìm kiếm từ khóa BM25-like, tìm kiếm ngữ nghĩa bằng embedding tiếng Việt, hợp nhất thứ hạng bằng RRF và xếp hạng lại bằng reranker. BM25 giúp giữ độ nhạy với thuật ngữ chính xác, còn embedding giúp bắt được các câu hỏi diễn đạt theo cách tự nhiên. RRF được dùng để hợp nhất hai danh sách kết quả mà không yêu cầu điểm số của chúng nằm trên cùng một

thang đo.

Trên lớp truy xuất cơ sở, đồ án xây dựng `AdaptiveRAGAgent`. Thành phần này chọn strategy dựa trên `QueryClassifier` heuristic, không dùng LLM để ra quyết định. Bốn strategy chính gồm STANDARD, BROAD, CURRICULUM và HIERARCHICAL. STANDARD dùng cho truy vấn cụ thể nhưng thiếu phạm vi rõ. BROAD dùng cho câu hỏi tổng quan. CURRICULUM ưu tiên metadata chương trình học. HIERARCHICAL dùng khi có đủ tín hiệu lớp/chủ đề để truy xuất theo cấp bài học rồi mới đi vào chunk chi tiết.

Hệ thống còn bổ sung scope fallback. Khi phạm vi sách/lớp chỉ đến từ UI và không đủ chunk phù hợp, `RAGService` có thể mở rộng phạm vi tìm kiếm, đồng thời ghi lại `actual_scope`, `scope_fallback_notice` và debug step tương ứng. Nhờ vậy, hệ thống không trả lời rỗng quá sớm nhưng vẫn minh bạch về phạm vi dữ liệu đã dùng.

5.2.3 Đóng góp đạt được

Đóng góp ở đây không chỉ là ghép nhiều kỹ thuật retrieval, mà là đặt chúng trong một cơ chế chọn strategy theo loại câu hỏi. Cách làm này phù hợp với bài toán giáo dục vì câu hỏi của học sinh và giáo viên có mức độ cụ thể rất khác nhau. Adaptive RAG giúp hệ thống cân bằng giữa độ chính xác, độ bao phủ và chi phí truy xuất.

Việc `QueryClassifier` là heuristic cũng là một quyết định có chủ đích. Thay vì gọi LLM thêm một lượt để chọn chiến lược truy xuất, hệ thống giữ bước này nhanh, có thể kiểm thử và có thể giải thích. Đây là điểm quan trọng trong một hệ thống cần ổn định hơn là thể hiện khả năng suy luận tự do.

5.3 Multi-intent routing và action planning

5.3.1 Vấn đề cần giải quyết

Trong giao diện hội thoại, người dùng không luôn đưa ra một yêu cầu đơn lẻ. Một câu có thể chứa nhiều ý định, ví dụ yêu cầu tạo slide rồi thêm câu hỏi luyện tập, hoặc vừa giải thích kiến thức vừa sinh bài tập. Nếu hệ thống chỉ chọn một intent duy nhất, một phần yêu cầu sẽ bị bỏ qua. Ngược lại, nếu để LLM tự thực thi toàn bộ chuỗi hành động, hệ thống dễ sinh ra luồng không nhất quán với các service đang có.

5.3.2 Giải pháp

`IntentRouter.detect_multi()` phân tích tối đa ba intent trong một câu hỏi. Mỗi intent gồm nhóm ý định chính, loại nhiệm vụ, chủ đề, lớp, bộ sách và một số metadata cần thiết. Sau đó, `ActionPlanner.plan_all()` chuyển danh

sách intent thành danh sách action cụ thể như sinh quiz, sinh slide, sinh giáo án, chấm câu trả lời, ôn câu sai hoặc giải thích khái niệm.

Việc giới hạn số intent là một quyết định thực tế. Nó đủ để xử lý các câu lệnh ghép thường gặp, nhưng tránh để một câu nhập mơ hồ sinh ra quá nhiều action và làm phình state. Sau bước lập kế hoạch, `ExecutionDispatcher` chịu trách nhiệm gọi đúng service hoặc handler theo action đã xác định.

5.3.3 Đóng góp đạt được

Cơ chế này giúp hệ thống xử lý yêu cầu tự nhiên tốt hơn mà vẫn giữ được tính quyết định của backend. LLM được dùng để hiểu câu nói của người dùng, nhưng kế hoạch thực thi cuối cùng được đưa về cấu trúc action rõ ràng. Điều này tạo ra một ranh giới cần thiết giữa năng lực ngôn ngữ của LLM và logic nghiệp vụ của phần mềm.

5.4 Tách quiz standalone khỏi content assessment

5.4.1 Vấn đề cần giải quyết

Sinh câu hỏi luyện tập có thể xuất hiện trong hai bối cảnh khác nhau. Bối cảnh thứ nhất là quiz standalone, nơi người dùng tạo bộ câu hỏi, trả lời, được chấm điểm, xem lại câu sai và theo dõi tiến độ. Bối cảnh thứ hai là assessment nhúng trong slide hoặc giáo án, nơi câu hỏi chỉ là một artifact phụ để củng cố bài giảng.

Nếu gộp hai bối cảnh này vào cùng một agent hoặc cùng một pipeline, hệ thống sẽ khó quản lý state. Quiz standalone cần lưu từng lượt làm bài, từng câu hỏi, câu trả lời của người dùng, số lần thử và kết quả chấm. Assessment trong slide lại chủ yếu cần nội dung phù hợp với bài học và format trình bày.

5.4.2 Giải pháp

Đề án giữ `QuizService` là một service độc lập cho quiz tương tác. Service này quản lý vòng đời sinh câu hỏi, kiểm tra chất lượng, lưu `QuizRound`, lưu `QuestionRecord`, chấm câu trả lời, review câu sai và thống kê tiến độ. Các handler câu hỏi như trắc nghiệm, tự luận, điền khuyết và đúng/sai được giữ trong pipeline quiz riêng.

Trong content multi-agent pipeline, `ContentAssessmentAgent` chỉ sinh assessment nhúng cho slide hoặc giáo án. Agent này không thay thế `QuizService` và không quản lý state làm bài của học sinh. Cách tách này giúp tránh nhầm lẫn giữa một bài kiểm tra tương tác và một phần hoạt động trong tài liệu giảng dạy.

5.4.3 Đóng góp đạt được

Đây là một đóng góp kiến trúc quan trọng vì nó giữ đúng ranh giới nghiệp vụ. Quiz standalone có thể phát triển thêm các chức năng học tập cá nhân hóa mà không ảnh hưởng đến pipeline sinh slide. Ngược lại, pipeline sinh slide/giáo án có thể tối ưu cho artifact dài mà không phải mang theo toàn bộ logic chấm điểm và theo dõi attempt.

5.5 Content multi-agent cho slide và giáo án

5.5.1 Vấn đề cần giải quyết

Tạo slide và giáo án là tác vụ dài hơn nhiều so với hỏi đáp hoặc sinh một bộ câu hỏi ngắn. Hệ thống phải xác định mục tiêu học tập, lập dàn ý, viết nội dung chi tiết, gợi ý media, thêm assessment, ghép kết quả và kiểm tra chất lượng. Nếu giao toàn bộ tác vụ này cho một prompt duy nhất, đầu ra dễ thiếu cấu trúc, bỏ sót phần quan trọng hoặc khó sửa khi người dùng muốn thay đổi dàn ý.

5.5.2 Giải pháp

Đồ án dùng LangGraph cho phạm vi content pipeline của slide và giáo án. Ở cấp hệ thống, Orchestrator vẫn là controller mã nguồn; LangGraph chỉ điều phối luồng sinh nội dung dài. ContentSupervisor gọi các tool adapter tương ứng với các specialist agent như PedagogyPlannerAgent, ContentDraftingAgent, MediaResearchAgent, ContentAssessmentAgent và QualityReviewerAgent.

Các agent không trao đổi tự do mà đi qua contract A2A-lite. Supervisor tạo AgentTask, specialist agent trả AgentTaskResult. Task chứa mục tiêu, đầu vào, ràng buộc, context refs và artifact mong đợi. Kết quả chứa trạng thái, artifact, confidence, warnings, used tools và latency. Contract này giúp mỗi kết quả trung gian có chủ sở hữu rõ ràng và dễ ghi debug.

Sau khi các artifact được tạo, merge_results ghép outline, content, media và assessment thành kết quả thống nhất. Thành phần merge là service/tool quyết định, không phải agent. Sau đó, QualityReviewerAgent đánh giá kết quả cuối và reflection_decision_node quyết định approve, revise hoặc block theo số vòng reflection cho phép.

5.5.3 Đóng góp đạt được

Đóng góp chính của phần này là tổ chức tác vụ sinh nội dung dài thành các bước có artifact rõ ràng. Hệ thống không chỉ sinh một đoạn văn lớn, mà có dàn ý, nội dung, media metadata, assessment và review. Điều này giúp slide và giáo án dễ kiểm soát hơn, đồng thời chuẩn bị nền tảng để sau này xuất sang PPTX, PDF hoặc

HTML.

A2A-lite cũng là một lựa chọn phù hợp với quy mô hiện tại. Hệ thống chưa cần runtime distributed agent đầy đủ, nhưng đã có message envelope đủ rõ để quản lý task, artifact và metadata thực thi. Cách làm này giảm độ phức tạp vận hành nhưng vẫn giữ đường nâng cấp trong tương lai.

5.6 Human-in-the-loop tại bước duyệt dàn ý

5.6.1 Vấn đề cần giải quyết

Trong sinh slide và giáo án, dàn ý là quyết định ảnh hưởng đến toàn bộ artifact phía sau. Nếu dàn ý sai trọng tâm, thiếu mục hoặc chia bài không hợp lý, các bước sinh nội dung, media và assessment sau đó đều bị ảnh hưởng. Sửa ở cuối pipeline thường tốn chi phí hơn nhiều so với sửa ngay sau khi có outline.

5.6.2 Giải pháp

Hệ thống đặt HITL tại bước outline review. Sau khi agent lập dàn ý, graph có thể interrupt để người dùng duyệt hoặc gửi feedback chỉnh sửa. SlideService lưu các thông tin resume như thread id, graph config, trạng thái interrupt và task type vào slide state. Khi người dùng xác nhận hoặc góp ý, pipeline resume từ điểm đã dừng thay vì chạy lại toàn bộ request từ đầu.

5.6.3 Đóng góp đạt được

HITL giúp đưa quyền kiểm soát vào đúng điểm có ảnh hưởng lớn nhất. Người dùng, đặc biệt là giáo viên, có thể điều chỉnh cấu trúc bài giảng trước khi hệ thống sinh nội dung chi tiết. Cách làm này phù hợp với đặc thù giáo dục, nơi người dùng thường muốn kiểm soát mục tiêu và mạch bài học thay vì chỉ nhận một kết quả hoàn chỉnh.

5.7 Quality path, session và trace

5.7.1 Vấn đề cần giải quyết

Một hệ thống LLM phục vụ giáo dục cần nhiều lớp kiểm soát hơn một chatbot thông thường. Câu hỏi sinh ra cần đúng format và bám ngữ cảnh. Câu trả lời cần lưu được theo phiên để người dùng hỏi tiếp. Các lỗi cần có trace để biết hệ thống đã chọn intent, action và strategy RAG nào.

5.7.2 Giải pháp

Đồ án xây dựng hai quality path riêng. Với quiz standalone, QualityReviewer kiểm tra chất lượng bộ câu hỏi và QuestionValidator kiểm tra từng câu hỏi trước khi lưu vào session. Nếu cần, handler có thể được gọi lại với revision instruction. Với slide và giáo án, kết quả sau merge được đưa qua QualityRe-

`viewerAgent`; graph có thể approve, revise một phần hoặc block nếu chất lượng chưa đạt.

Session được lưu bằng `SessionStore` dưới dạng JSON. Session không chỉ lưu lịch sử chat mà còn lưu quiz state và slide state, cho phép các yêu cầu nối tiếp như “chấm câu 2”, “ôn câu sai” hoặc “ok, duyệt dần ý”. Song song với session, `TraceService` và `RequestContext.add_debug_step()` ghi lại các bước xử lý quan trọng để hỗ trợ kiểm thử và gỡ lỗi.

5.7.3 Đóng góp đạt được

Các lớp quality, session và trace làm cho hệ thống có khả năng vận hành thực tế hơn. Hệ thống không chỉ tạo được câu trả lời, mà còn quản lý được vòng đời học tập của người dùng và cung cấp dữ liệu quan sát khi kết quả chưa đạt kỳ vọng. Đây là nền tảng cần thiết để tiếp tục cải thiện chất lượng truy xuất, đánh giá và cá nhân hóa trong các phiên bản sau.

5.8 Tool layer như điểm mở rộng có kiểm soát

5.8.1 Vấn đề cần giải quyết

Một số năng lực của hệ thống không nên được nhúng trực tiếp vào prompt hoặc agent, ví dụ truy xuất tri thức, định dạng context, tìm kiếm media, kiểm tra URL hoặc xuất artifact. Nếu mỗi nơi gọi tool theo một kiểu riêng, hệ thống sẽ khó mở rộng và khó thay thế implementation.

5.8.2 Giải pháp

Đồ án xây dựng một MCP-like tool layer in-process. Lớp này gồm protocol request/response, client, server và registry tool. Các tool như knowledge retrieval hoặc content formatting có thể được gọi qua interface thống nhất. Hiện tại, lớp này chủ yếu đóng vai trò chuẩn hóa tool call nội bộ và làm extension point; chưa được mô tả như một MCP phân tán hoàn chỉnh.

5.8.3 Đóng góp đạt được

Tool layer giúp tách phần năng lực mở rộng khỏi core orchestration. Khi cần bổ sung media search thật, URL validation, source attribution hoặc export artifact, hệ thống đã có vị trí phù hợp để tích hợp mà không làm thay đổi sâu các service hiện có. Đây là một đóng góp về khả năng mở rộng hơn là một tính năng hoàn chỉnh ở thời điểm hiện tại.

5.9 Kết chương

Chương này đã hệ thống hóa các giải pháp và đóng góp chính của đồ án. Các đóng góp không nằm ở một thuật toán riêng lẻ, mà ở cách kết hợp LLM với các lớp điều phối, truy xuất, kiểm tra chất lượng, quản lý trạng thái và quan sát hệ thống.

Thiết kế này giúp Educational Chatbot tránh phụ thuộc hoàn toàn vào khả năng sinh tự do của LLM, đồng thời giữ được tính mở rộng cho các chức năng giáo dục như quiz, slide và giáo án.

Những đóng góp trên cũng cho thấy các khó khăn chính của đề tài đã được xử lý theo hướng thực dụng: truy xuất phải thích nghi với loại câu hỏi, điều phối phải kiểm soát được nhiều intent, quiz tương tác phải tách khỏi artifact generation, multi-agent phải có contract rõ ràng, và người dùng cần được đưa vào vòng duyệt ở các điểm quyết định quan trọng.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Sinh viên so sánh kết quả nghiên cứu hoặc sản phẩm của mình với các nghiên cứu hoặc sản phẩm tương tự.

Sinh viên phân tích trong suốt quá trình thực hiện ĐATN, mình đã làm được gì, chưa làm được gì, các đóng góp nổi bật là gì, và tổng hợp những bài học kinh nghiệm rút ra nếu có.

6.2 Hướng phát triển

Trong phần này, sinh viên trình bày định hướng công việc trong tương lai để hoàn thiện sản phẩm hoặc nghiên cứu của mình.

Trước tiên, sinh viên trình bày các công việc cần thiết để hoàn thiện các chức năng/nhiệm vụ đã làm. Sau đó sinh viên phân tích các hướng đi mới cho phép cải thiện và nâng cấp các chức năng/nhiệm vụ đã làm.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thức của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

[1] E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

[2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

[3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

[4] M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

[5] A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

[6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available:

`ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

[7] Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO

- [1] E. H. Hovy, “Automated discourse generation using discourse structure relations,” *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993.
- [2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.
- [3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.
- [4] M. Poesio and B. Di Eugenio, “Discourse structure and anaphoric accessibility,” in *ESSLLI workshop on information structure, discourse structure and discourse semantics, Copenhagen, Denmark*, 2001, pp. 129–143.
- [5] A. Knott, “A data-driven methodology for motivating a set of coherence relations,” Ph.D. dissertation, The University of Edinburgh, UK, 1996.
- [6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. Accessed: Sep. 30, 2010. [Online]. Available: <ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z>.
- [7] Princeton University, *Wordnet*. Accessed: Sep. 30, 2010. [Online]. Available: <http://www.cogsci.princeton.edu/~wn/index.shtml>.

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template ĐATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ ĐATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong ĐATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày ĐATN theo kiểu viết ý hoặc gạch đầu dòng. ĐATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết ĐATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa

học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi trong từng trang báo cáo DATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

1. Đối với kỹ sư chính quy:
 - (a) Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
 - (b) Từ K62 trở về sau: Ngành Khoa học máy tính
2. Đối với cử nhân:
 - (a) Ngành Công nghệ thông tin
3. Đối với chương trình EliteTech:
 - (a) Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
 - (b) Chương trình ICT Global: Ngành Information Technology
 - (c) Chương trình DS&AI: Ngành Khoa học dữ liệu

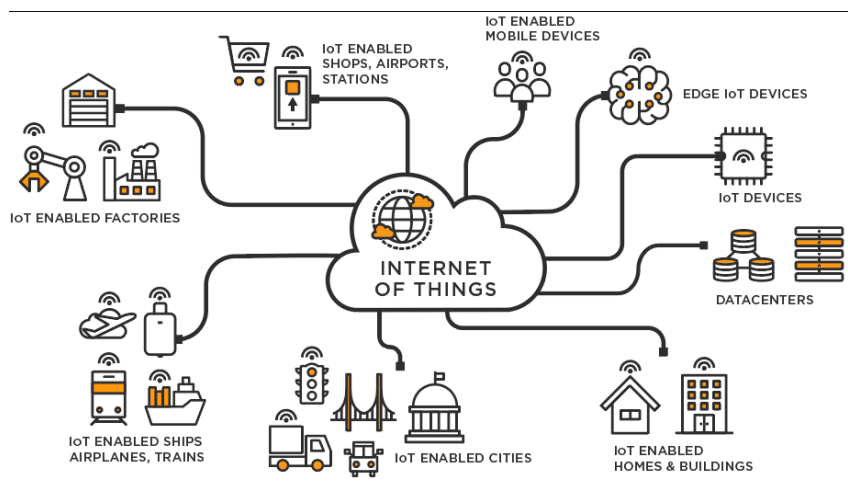
A.2 Đánh dấu (bullet) và đánh số (numering)

Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```
- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
```



Hình A.1: Internet vạn vật

```
\item ...
\end{enumerate}
```

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.3 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong LaTeX khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau [2]. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau [3]. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

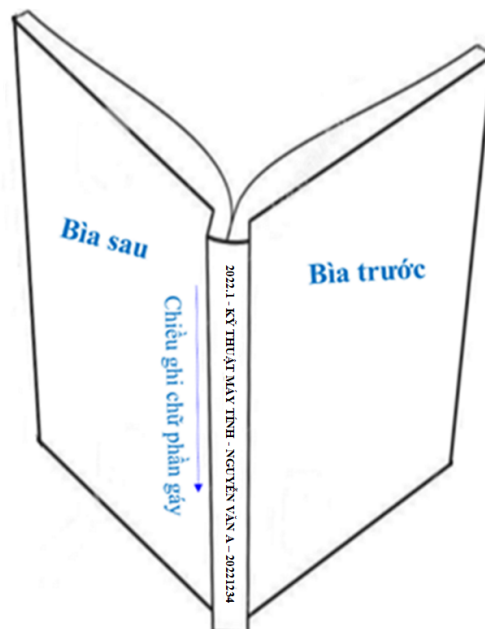
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.3 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Qui cách đóng quyển đồ án



Hình A.3: Qui cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case “Tổng kê tình hình mượn sách”

...

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...